Quaderno AI

Concetti, strumenti e percorsi pratici per lo sviluppo software con AI

Andrea Merlin

Numero 1 - Novembre 2026

Indice

Capitolo 1 **1**

Model Context Protocol: fondamenti e implementazione di un MCP Server in .NET 10 1

1.1 Il problema: modelli intelligenti ma isolati 2

1.2 Che cos'è il Model Context Protocol 4

1.3 Architettura: Host, Client e Server 6

1.4 I primitivi del protocollo: Tools, Resources e Prompts 9

1.5 JSON-RPC 2.0: il linguaggio del protocollo 11

1.6 I trasporti: stdio, Streamable HTTP e SSE 14

1.7 Il ciclo di vita di una sessione MCP 16

1.8 MCP vs REST API: un confronto approfondito 18

1.9 L'ecosistema .NET: pacchetti e namespaces 22

1.10 Preparare l'ambiente di sviluppo 24

1.11 Il tuo primo MCP server (stdio) 26

1.12 I Tools in dettaglio 30

1.13 Esempio pratico: un server per il file system 33

1.14 Esempio pratico: un server meteo con HTTP 37

1.15 Le Risorse 40

1.16 I Prompt 42

1.17 Server HTTP con ASP.NET Core 44

1.18 Configurare i client: Claude Desktop e VS Code 47

1.19 Testare con MCP Inspector 49

1.20 Errori, logging e best practice 51

1.21 Sicurezza e operatività 58

1.22 Riepilogo, glossario ed esercizi 60

Capitolo 1

Model Context Protocol: fondamenti e implementazione di un MCP Server in .NET 10

Abstract. I modelli linguistici di grandi dimensioni (Large Language Model, LLM) sono strumenti potenti ma, presi da soli, "isolati": non possono accedere ai tuoi file, alle tue API, ai tuoi database o agli strumenti che usi ogni giorno. Il *Model Context Protocol* (MCP) è lo standard aperto che risolve questo problema, definendo un modo uniforme con cui un'applicazione basata su intelligenza artificiale può scoprire e usare **strumenti**, **dati** e **modelli di prompt** esposti da un processo chiamato *MCP server*. In questo capitolo capiremo perché MCP è nato, in che cosa si distingue da una classica API REST, e costruiremo passo dopo passo, con codice reale e funzionante, un MCP server completo in .NET 10.

Cosa imparerai in questo capitolo

Al termine della lettura e della pratica saprai:

- spiegare a parole semplici cos'è il Model Context Protocol e quale problema risolve;
- distinguere con chiarezza un MCP server da una REST API e scegliere lo strumento giusto per ogni scenario;
- descrivere l'architettura del protocollo (host, client, server) e i suoi primitivi (tools, resources, prompts);
- leggere e scrivere i messaggi JSON-RPC 2.0 che costituiscono il "linguaggio" di MCP;
- creare da zero un progetto .NET 10 e aggiungere i pacchetti NuGet ufficiali del C# SDK;
- implementare un MCP server su trasporto `stdio` e su trasporto HTTP con ASP.NET Core;
- esporre tool con attributi C#, gestire parametri, descrizioni, codice asincrono ed errori;

- esporre risorse (resources) e template di prompt (prompts);
- configurare client reali come Claude Desktop e Visual Studio Code;
- collaudare il server con l'MCP Inspector;
- applicare le principali best practice di sicurezza e di progettazione.

Prerequisiti

- Conoscenza di base del linguaggio C# e dell'ecosistema .NET;
- .NET SDK 10 installato (vedremo come verificarlo e installarlo nella sezione dedicata);
- un editor di testo o un IDE (Visual Studio Code, Visual Studio o Rider);
- facoltativa ma consigliata: familiarità con il formato JSON e con il concetto di HTTP.

Convenzioni tipografiche

Nel corso del capitolo useremo alcune convenzioni ricorrenti:

- Il **grassetto** evidenzia concetti nuovi o importanti.
- Il testo `monospace` indica nomi di pacchetti, tipi, metodi, comandi e frammenti di codice nel testo.
- I blocchi di codice riportano il linguaggio nell'intestazione (per esempio `csharp`, `json`, `bash`).
- I comandi da eseguire in un terminale sono preceduti dal prompt `>` e vanno eseguiti nella cartella indicata.
- Le note *Importante* e *Attenzione* segnalano rispettivamente un concetto da ricordare e una trappola comune in cui è facile cadere.

Ogni frammento di codice di questo capitolo è stato **compilato ed eseguito** con .NET 10 e con la versione 2.3.0 dei pacchetti ufficiali del C# SDK: puoi quindi copiarlo e incollarlo con fiducia. Dove la versione del pacchetto è rilevante, la indicheremo esplicitamente, perché le API dell'SDK sono in rapida evoluzione.

1.1 Il problema: modelli intelligenti ma isolati

Immagina di dover costruire un assistente che risponde a domande sui dati della tua azienda: "quante fatture sono state emesse questa settimana?", "riassumi i commenti dei clienti sul prodotto X", "aggiorna lo stato della pratica 1234". Un LLM, da solo, non può farlo. Il modello conosce il linguaggio, sa ragionare e sa produrre testo, ma:

- non può **leggere** il tuo database, i tuoi file o le tue API;
- non può **agire** — cioè compiere operazioni come creare un ticket, inviare una email o aggiornare un record;
- non ha **memoria persistente** tra una sessione e l'altra: ogni conversazione riparte da zero;

- non conosce il **contesto specifico** della tua organizzazione, che non fa parte del suo addestramento.

La soluzione tradizionale è stata per anni l'integrazione "a mano": ogni volta che nasceva una nuova applicazione di intelligenza artificiale, si scrivevano connettori su misura verso ogni sorgente dati, e viceversa. È quello che viene chiamato il **problema M×N**: con M applicazioni AI e N sorgenti di dati o strumenti, servono potenzialmente $M \times N$ integrazioni, ognuna con la propria logica, i propri formati e i propri bug.

Se un team costruisce un assistente che deve parlare con un CRM, un database SQL, un file system e un servizio meteo, e poi un secondo team ne costruisce un altro che deve parlare con lo stesso CRM e lo stesso database, ciascun team riscriverà da capo (o quasi) il codice di connessione. Il risultato è duplicazione, costi crescenti e una qualità disomogenea.

1.1.1 L'analogia con le prese di corrente

Un'analogia efficace è quella delle prese elettriche. Prima della standardizzazione, ogni elettrodomestico aveva il suo tipo di spina e ogni stanza la sua presa: per collegare un nuovo dispositivo serviva un adattatore. La standardizzazione della spina ha permesso a qualsiasi apparecchio di funzionare in qualsiasi presa, una sola volta per tutte.

Il Model Context Protocol fa la stessa cosa per il mondo dell'IA generativa: definisce una **"presa standard"** a cui ogni applicazione AI può collegarsi, e una **"spina standard"** che ogni fornitore di dati o strumenti può esporre. Una volta implementata la presa, tutte le spine sono compatibili; una volta implementata la spina, tutte le prese funzionano.

1.1.2 Cosa serve davvero a un LLM per essere utile

Possiamo riassumere le esigenze di un'applicazione LLM in tre categorie:

1. **Strumenti (tools)** — capacità di compiere azioni: eseguire una query, creare un file, inviare una richiesta HTTP. Il modello decide *quando* e *come* usarli, in base alla richiesta dell'utente.
2. **Dati (resources)** — contenuti informativi da inserire nel contesto: documenti, record di database, configurazioni, log. Il modello li *legge* per rispondere meglio.
3. **Istruzioni riutilizzabili (prompts)** — modelli di prompt predefiniti che l'utente o il client possono richiamare, per non riscrivere ogni volta le stesse istruzioni.

Un LLM dotato di questi tre ingredienti smette di essere un semplice "generatore di testo" e diventa un **agente**: un sistema che osserva, ragiona e agisce.

1.1.3 Perché serve un *protocollo* e non una libreria

A questo punto potresti chiederti: perché non usare semplicemente una libreria client per parlare con il database o le API? In effetti è possibile, e in molti casi si fa. Il problema è che ogni libreria ha la propria sintassi, i propri tipi, i propri errori. Quando l'applicazione AI deve imparare a usare un nuovo strumento, deve "imparare" anche la sua API specifica.

Un **protocollo** standardizza invece l'intera conversazione: come si scoprono gli strumenti disponibili, come si invoca uno strumento, come si ricevono i risultati, come si gestiscono gli errori. L'applicazione AI non deve più conoscere i dettagli di ogni singola sorgente: conosce il protocollo, e tutto ciò che parla il protocollo è automaticamente utilizzabile.

Questa è la stessa idea che ha reso di successo, in altri campi, protocolli come HTTP per il web, SMTP per la posta o il Language Server Protocol (LSP) per gli editor di codice: definire un **contratto unico** e lasciare che il resto del mondo vi si adegui.

> **Nota.** MCP non sostituisce il *funzionamento interno* dei tuoi sistemi. Il tuo database continuerà a usare SQL, le tue API REST continueranno a esporre endpoint HTTP. MCP è un **livello di interfaccia standard** che si appoggia sopra questi sistemi e li rende consumabili da un LLM in modo uniforme. Torneremo su questa distinzione fondamentale nella sezione dedicata al confronto con le REST API.

1.2 Che cos'è il Model Context Protocol

Il **Model Context Protocol (MCP)** è un protocollo aperto e libero (open source) che standardizza il modo in cui le applicazioni forniscono **contesto** ai modelli linguistici. In termini concreti, MCP definisce:

- un **formato di messaggi** basato su JSON-RPC 2.0 con cui client e server comunicano;
- un insieme di **operazioni** standard (elenca gli strumenti, invoca uno strumento, leggi una risorsa, ottieni un prompt, ecc.);
- un modello di **capabilities** (capacità) che client e server dichiarano e negoziano;
- una famiglia di **trasporti** (canali di comunicazione) su cui i messaggi viaggiano: il più comune è `stdio`, seguito da HTTP.

La definizione ufficiale, ripresa dai documenti del progetto, è particolarmente chiara:

> Il Model Context Protocol è un protocollo aperto che standardizza il modo in cui le applicazioni forniscono contesto ai Large Language

Model (LLM). Consente un'integrazione sicura tra gli LLM e le varie sorgenti di dati e strumenti.

1.2.1 Chi c'è dietro MCP

MCP è stato **annunciato da Anthropic nel novembre 2024**, insieme ai primi SDK ufficiali, tra cui quelli per TypeScript e Python. L'obiettivo dichiarato era duplice: da un lato semplificare la vita agli sviluppatori di applicazioni basate su Claude, dall'altro offrire alla comunità un unico standard aperto invece di una miriade di formati proprietari.

Fin dall'inizio MCP è stato pubblicato con licenza **MIT** e con una specifica pubblica, ospitata nell'organizzazione GitHub `modelcontextprotocol`. Questo ha permesso un'adozione rapida e trasversale: oggi esistono SDK ufficiali o mantenuti dalla comunità per decine di linguaggi (C#, Java, Go, Rust, Kotlin, Swift, PHP e molti altri), e migliaia di server MCP pubblici per servizi come GitHub, Google Drive, PostgreSQL, Slack, Spotify e così via.

Oltre ad Anthropic, numerosi attori dell'ecosistema hanno annunciato supporto a MCP: OpenAI, Microsoft, Google (con Gemini), AWS e molti strumenti di sviluppo. Visual Studio Code e GitHub Copilot, in particolare, supportano nativamente la configurazione di server MCP, come vedremo più avanti.

1.2.2 Gli obiettivi di progetto

Dalla specifica emergono alcuni principi guida che conviene tenere a mente mentre si progetta un server:

- **Semplicità.** Il protocollo deve essere semplice da implementare e da comprendere. Non è richiesto alcun framework complesso: bastano un canale di I/O e un parser JSON.
- **Riusabilità.** Un server MCP scritto una volta deve poter essere usato da qualsiasi client MCP, senza modifiche.
- **Sicurezza.** Il protocollo definisce meccanismi di autenticazione e autorizzazione (incluso OAuth 2.1 per i trasporti HTTP) e incoraggia il principio del minimo privilegio.
- **Estensibilità.** Le capabilities permettono di evolvere il protocollo senza rompere la compatibilità.
- **Simmetria.** Il protocollo è bidirezionale: non solo il client chiede e il server risponde, ma anche il server può fare richieste al client (per esempio per il *sampling*, cioè chiedere al modello di generare testo).

1.2.3 Un server, molti client

La promessa centrale di MCP si riassume in una frase: **scrivi una volta, usa ovunque**. Un server MCP che espone, poniamo, gli strumenti del tuo CRM viene "capito" da:

- Claude Desktop, l'applicazione desktop di Anthropic;
- Visual Studio Code con GitHub Copilot;
- qualsiasi client che implementi la specifica, sia esso un chatbot, un agente autonomo, un'estensione di un IDE o un tool da riga di comando.

Il server non sa nulla del client che lo sta usando: non conosce il nome del modello, non sa se il chiamante è un utente umano o un agente autonomo. Il server espone una *descrizione* dei propri strumenti e dati, e il client (con il suo LLM) decide come usarli. Questa separazione netta delle responsabilità è il cuore del design.

1.2.4 MCP e LSP: una parentela naturale

Se hai mai sviluppato un'estensione per un editor di codice, avrai probabilmente incontrato il **Language Server Protocol (LSP)**, lo standard che permette a VS Code (e ad altri editor) di offrire completamento, refactoring e diagnostica per qualsiasi linguaggio. MCP è, per molti versi, il "cugino" di LSP per il mondo dei modelli:

- LSP standardizza la comunicazione tra un *editor* e un *language server*;
- MCP standardizza la comunicazione tra un'*applicazione AI* e un *server di contesto*.

Entrambi nascono da un'intuizione comune: invece di far parlare ogni client con ogni server tramite integrazioni ad hoc, si definisce un unico contratto. In entrambi i casi il trasporto più comune è `stdio`, ed entrambi usano messaggi JSON-RPC. La differenza sostanziale è l'oggetto della comunicazione: LSP riguarda il codice sorgente (diagnostica, simboli, riferimenti), MCP riguarda strumenti e dati generici messi a disposizione di un modello.

1.3 Architettura: Host, Client e Server

L'architettura di MCP distingue tre ruoli, che è fondamentale non confondere:

- **Host** — l'applicazione che l'utente vede e usa: Claude Desktop, VS Code, un'app mobile, un agente autonomo. L'host è responsabile dell'esperienza utente e, di norma, ospita il modello linguistico.
- **Client** — il componente interno all'host che gestisce la comunicazione con un singolo server. L'host può contenere **molti client**, uno per ogni server MCP a cui è collegato.
- **Server** — il processo che espone strumenti, risorse e prompt. È il componente che implementeremo in questo capitolo.

La relazione tra i tre ruoli può essere riassunta così:

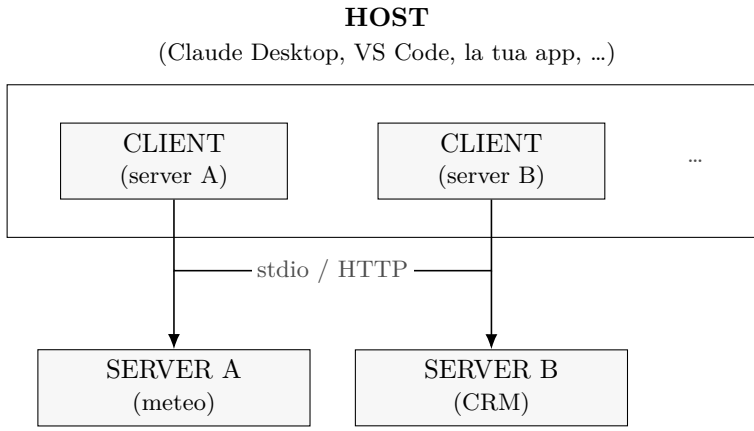


Figura 1: Architettura di MCP: un host, uno o più client (uno per server) e i server che espongono le capacità

Ogni client è legato a **un solo server**: la specifica prevede una relazione 1:1 tra client e server all'interno della sessione. L'host, invece, aggrega più client e orchestra il modello, passandogli i risultati dei vari server sotto forma di messaggi.

1.3.1 Cosa fa l'host

L'host è il regista della conversazione:

1. avvia i client e, tramite loro, i server (nel caso di `stdio` li lancia come processi figli);
2. raccoglie le capabilities dei server (quali tool, risorse e prompt offrono);
3. presenta questi strumenti al modello linguistico (tipicamente iniettandone le descrizioni nel prompt di sistema);
4. quando il modello decide di chiamare uno strumento, l'host inoltra la richiesta al client giusto;
5. restituisce al modello il risultato, permettendogli di continuare il ragionamento.

Nota bene: **il server non parla mai direttamente con il modello**. È l'host che media. Il server si limita a esporre funzioni e dati; è il modello, tramite l'host, a decidere se e quando usarli.

1.3.2 Cosa fa il client

Il client è un componente "meccanico": gestisce la connessione, serializza e deserializza i messaggi JSON-RPC, mantiene lo stato della sessione (handshake, negoziazione delle capabilities) e inoltra le risposte all'host. Negli SDK, il client

è quasi sempre una classe già pronta: per noi, nel ruolo di sviluppatori del server, il client è soprattutto il "chiamante" che invierà i messaggi.

1.3.3 Cosa fa il server

Il server è il nostro oggetto di studio. Le sue responsabilità sono:

- dichiarare le proprie **capabilities** durante l'handshake iniziale;
- rispondere alle richieste di **discovery** (elenco dei tool, delle risorse, dei prompt);
- **eseguire** le operazioni richieste (invocare un tool, leggere una risorsa, comporre un prompt);
- restituire risultati strutturati, eventualmente con contenuti testuali e immagini;
- notificare al client eventuali **eventi** asincroni (per esempio il cambiamento di una risorsa).

Nel C# SDK gran parte di questo lavoro è automatizzato: come vedremo, per esporre un tool basterà marcare un metodo C# con un attributo. L'SDK si occupa di introspezione, serializzazione, discovery e gestione del trasporto.

1.3.4 La negoziazione delle capabilities

MCP è un protocollo *capace di evolvere*: non tutti i client e i server supportano tutte le funzionalità. Per questo, durante l'handshake, client e server si scambiano le rispettive **capabilities**, cioè un elenco dichiarativo di ciò che sanno fare.

Il client dichiara, tra le altre cose:

- se supporta il **sampling** (permettere al server di chiedere generazione al modello);
- se supporta le **roots** (informare il server sulle cartelle o sugli ambiti di lavoro);
- se supporta la gestione di **tool, risorse e prompt**.

Il server dichiara:

- se espone **tool** (e con quali funzionalità);
- se espone **risorse** (e se supporta le sottoscrizioni ai cambiamenti);
- se espone **prompt**;
- se supporta il **logging** (inviare messaggi di log al client);
- eventuali estensioni sperimentali.

Il risultato è che un server molto semplice (solo tool) e un client molto semplice (solo tool) possono comunque comunicare correttamente, ignorando le funzionalità non condivise. Quando, più avanti, useremo l'SDK .NET, vedremo che la negoziazione avviene in modo automatico: a noi basterà dichiarare ciò che il server espone, e l'SDK costruirà la risposta di inizializzazione corretta.

1.4 I primitivi del protocollo: Tools, Resources e Prompts

Il cuore di MCP è costituito da tre primitivi, a cui se ne aggiungono alcuni secondari. Comprenderli a fondo è indispensabile, perché tutto il codice che scriveremo ruota attorno a essi.

1.4.1 Tools: le azioni

Un **tool** è una funzione che il server mette a disposizione del modello. Quando il modello "chiama" un tool, invia al server un nome e una lista di argomenti; il server esegue la funzione e restituisce un risultato.

Ogni tool è descritto da:

- un **nome** (stringa, univoco all'interno del server);
- una **descrizione** testuale, che il modello usa per capire *quando* e *come* usarlo: una buona descrizione è il fattore che più influenza la qualità dell'uso;
- uno **schema JSON** degli argomenti in ingresso (nome, tipo, obbligatorietà, descrizione di ogni parametro);
- opzionalmente, uno schema dell'output.

Esempi tipici di tool: `search_documents`, `send_email`, `create_ticket`, `get_weather`, `read_file`, `run_query`. Un tool può essere **idempotente** o meno, **che modifica lo stato** o di sola lettura: MCP prevede annotazioni facoltative per dichiararlo (ci torneremo).

È importante capire la direzione del flusso: nei tool è il **modello** a prendere l'iniziativa. Il server espone il menu, il modello sceglie dal menu e il server esegue.

1.4.2 Resources: i dati

Una **resource** è un dato esposto dal server e identificato da un **URI**. Se i tool sono le "azioni", le risorse sono i "documenti": file, record, configurazioni, immagini, log. Il modello (o il client, per conto dell'utente) può *leggere* una risorsa per acquisire contesto.

Le risorse hanno una differenza concettuale fondamentale rispetto ai tool: nei tool il controllo è del modello, nelle risorse il controllo è **del client o dell'applicazione**. Un'applicazione può decidere autonomamente di "agganciare" una risorsa al contesto del modello, senza che il modello debba chiederla.

Le risorse possono essere:

- **statiche**, con un URI fisso (per esempio `config://app/settings`);

- **parametrizzate tramite template**, in cui l'URI contiene dei segnaposto (per esempio `file://docs/{id}`): il client può elencare i template e poi leggere una risorsa concreta sostituendo i parametri.

Ogni risorsa dichiara un **tipo MIME** (per esempio `text/plain`, `application/json`, `image/png`), così il client sa come interpretarla.

1.4.3 Prompts: i modelli di istruzione

Un **prompt** è un modello di istruzioni riutilizzabile, esposto dal server e richiamabile dal client o dall'utente. Mentre un tool *esegue* qualcosa, un prompt *produce testo* (o una sequenza di messaggi) pronto per essere inviato al modello.

Un caso d'uso tipico: il server espone un prompt `review_code` che, dato un frammento di codice, produce il testo "Fai una revisione approfondita del seguente codice, segnalando bug, problemi di sicurezza e suggerimenti di stile: …". L'utente, dall'interfaccia dell'host, può selezionare il prompt senza doverlo riscrivere.

I prompt possono dichiarare **argomenti** (con schema JSON), e la loro esecuzione restituisce una lista di messaggi (ruolo + contenuto), non necessariamente una singola stringa.

1.4.4 Gli altri primitivi

Oltre ai tre principali, la specifica prevede primitivi più avanzati, che non implementeremo in dettaglio ma che è bene conoscere:

- **Sampling** — permette al *server* di chiedere al *client* di generare testo con il modello. Utile per scenari in cui il server deve, per esempio, riassumere un documento.
- **Roots** — permette al client di comunicare al server quali sono le "radici" del progetto (cartelle, URL) su cui l'utente sta lavorando.
- **Elicitation** — permette al server di porre una domanda all'utente durante l'esecuzione di un tool.
- **Logging** — permette al server di inviare messaggi di log strutturati al client.
- **Completions** — permette al server di suggerire completamenti per gli argomenti dei prompt.
- **Pagination e Progress** — per gestire elenchi lunghi e operazioni lente.

1.4.5 La tabella riassuntiva

Primitivo	Direzione del controllo	Scopo	Esempio
Tool	Modello	Eseguire azioni	`create_ticket`, `get_weather`
Resource	Client / applicazione	Fornire dati di contesto	`config://app`, `docs://123`
Prompt	Utente / client	Riutilizzare istruzioni	`review_code`, `summarize_meeting`

Tenere a mente questa tabella aiuta a scegliere il primitivo giusto quando si progetta un server: se il modello deve *fare* qualcosa, serve un tool; se deve *leggere* qualcosa, una resource; se l'utente deve *invocare un'istruzione pronta*, un prompt.

1.5 JSON-RPC 2.0: il linguaggio del protocollo

Tutta la comunicazione MCP avviene tramite messaggi **JSON-RPC 2.0**. JSON-RPC è un protocollo remoto estremamente semplice: un messaggio JSON che descrive una chiamata di procedura remota, con un identificatore per correlare richieste e risposte. Se hai mai lavorato con RPC, gRPC o con le API JSON di Ethereum, il concetto ti sarà familiare.

MCP usa JSON-RPC 2.0 con alcune convenzioni specifiche. I tipi di messaggio sono tre: request, notification e response; l'errore è una variante della response.

1.5.1 La richiesta (request)

Una richiesta è un messaggio inviato da una parte per chiedere all'altra di eseguire un'operazione. Ha un campo `method` e, opzionalmente, un campo `params`:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "echo",
    "arguments": { "message": "ciao" }
  }
}
```

Il campo `id` può essere una stringa o un numero e serve a correlare la risposta. Quando l'operazione non richiede risposta, il messaggio diventa una *notifica* (vedi sotto) e l'`id` viene omesso.

1.5.2 La risposta (response)

La risposta a una richiesta riporta lo stesso `id` e il risultato nel campo `result`:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [
      { "type": "text", "text": "hello ciao" }
    ],
    "isError": false
  }
}
```

1.5.3 La variante di errore della risposta

In caso di errore a livello di protocollo, la risposta contiene un campo `error` al posto di `result`. Non è un quarto tipo di messaggio, ma una variante della response:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Invalid params"
  }
}
```

I codici di errore standard di JSON-RPC, che MCP riusa, sono:

Codice	Significato
-32700	Parse error
-32600	Invalid request
-32601	Method not found
-32602	Invalid params
-32603	Internal error

A questi MCP aggiunge codici specifici, come `-32002` (resource not found) o `-32022` (unsupported protocol version). Nell'SDK .NET questi valori sono raccolti nell'enumerazione `McpErrorCode`, che useremo nella sezione sulla gestione degli errori.

Un errore durante l'esecuzione di un tool è diverso: `tools/call` restituisce comunque un `result`, il cui `CallToolResult` segnala `isError: true`.

1.5.4 La notifica (notification)

Una notifica è una richiesta *senza* `id`: il destinatario la esegue ma non deve rispondere. Si usa per eventi unidirezionali. L'esempio più importante è la notifica di inizializzazione completata:

```
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized",
  "params": {}
}
```

1.5.5 I metodi standard di MCP

Sopra JSON-RPC, MCP definisce una serie di metodi con nomi a barre (`risorsa/azione`). I più importanti, lato server:

Metodo	Scopo
`initialize`	Avvia la sessione e negozia le capabilities
`notifications/initialized`	Notifica che l'inizializzazione è conclusa
`ping`	Verifica che la connessione sia viva
`tools/list`	Elenca i tool disponibili
`tools/call`	Invoca un tool
`resources/list`	Elenca le risorse
`resources/read`	Legge una risorsa
`resources/templates/list`	Elenca i template di risorsa
`prompts/list`	Elenca i prompt
`prompts/get`	Ottiene un prompt
`logging/setLevel`	Imposta il livello di log
`completion/complete`	Richiede completamenti

Il bello di lavorare con l'SDK .NET è che **non dovremo mai scrivere questi messaggi a mano**: l'SDK li genera e li interpreta per noi. Conoscerli, però, è essenziale per il debugging, per leggere i log e per capire cosa succede "sotto il cofano". Inoltre, quando useremo l'MCP Inspector o un client generico, sarà naturale ragionare in termini di questi metodi.

1.5.6 Un esempio completo di scambio

Mettiamo insieme i pezzi per vedere una sessione minima. Il client apre la connessione e invia l'inizializzazione:

```
{"jsonrpc":"2.0","id":1,"method":"initialize","params":{
  "protocolVersion":"2025-06-18",
  "capabilities":{},
  "clientInfo":{"name":"book-test","version":"1.0"}
}}
```

Il server risponde con le proprie informazioni e capacità:

```
{"jsonrpc":"2.0","id":1,"result":{
  "protocolVersion":"2025-06-18",
  "capabilities":{"tools":{}},
  "serverInfo":{"name":"McpTest","version":"1.0.0.0"}
}}
```

Il client notifica che l'inizializzazione è completa e poi elenca i tool:

```
{"jsonrpc":"2.0","method":"notifications/initialized","params":{}}
{"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}
```

Il server risponde con l'elenco (nota il nome in minuscolo, un dettaglio dell'SDK che documenteremo):

```
{"jsonrpc":"2.0","id":2,"result":{"tools":[
  {"name":"echo","description":"Echoes the message back to the client.",
   "inputSchema":{"type":"object","properties":{
      "message":{"type":"string","description":"The message to echo."}},
    "required":["message"]}}
]}}
```

Infine il client invoca il tool:

```
{"jsonrpc":"2.0","id":3,"method":"tools/call","params":{
  "name":"echo","arguments":{"message":"mondo"}}}
```

E il server restituisce il risultato:

```
{"jsonrpc":"2.0","id":3,"result":{
  "content":[{"type":"text","text":"hello mondo"}],
  "isError":false
}}
```

Questo scambio, pur minimale, contiene tutto il DNA di MCP. Lo rivedremo realizzato davvero, byte per byte, quando eseguiremo il nostro primo server.

1.6 I trasporti: stdio, Streamable HTTP e SSE

JSON-RPC definisce *cosa* si dice, ma non *come* lo si trasporta. MCP definisce una famiglia di **trasporti**: canali concreti su cui i messaggi viaggiano. La scelta

del trasporto è una delle decisioni progettuali più importanti, perché determina dove e come il server verrà eseguito.

1.6.1 stdio: il trasporto dei processi locali

Nel trasporto `stdio`, il server viene **lanciato come processo figlio** dal client, e la comunicazione avviene attraverso lo standard input e lo standard output del processo:

- il client scrive i messaggi JSON-RPC sullo **stdin** del server;
- il server risponde sullo **stdout**;
- i messaggi sono delimitati da **newline** (un oggetto JSON per riga).

È il trasporto più semplice e più diffuso per i server locali: non richiede porte di rete, né configurazioni di firewall, né esposizione a Internet. Il ciclo di vita del server coincide con quello del processo: quando il client si chiude (o lo stdin raggiunge la fine), il server termina.

La conseguenza pratica più importante, che vale la pena evidenziare subito, è questa: **su `stdio` lo stdout appartiene al protocollo**. Qualsiasi altra scrittura sullo stdout — per esempio i log della console — corrompe il flusso JSON-RPC e rende il server inutilizzabile. Per questo, nei nostri server `stdio`, disabiliteremo il logging su console con `builder.Logging.ClearProviders()`.

1.6.2 Streamable HTTP: il trasporto di rete

Il trasporto **Streamable HTTP** è il trasporto di rete raccomandato dalla specifica moderna. Il server espone un **singolo endpoint HTTP** (per esempio `http://localhost:8080/mcp`) e:

- il client invia le richieste JSON-RPC con normali richieste `POST`;
- il server può rispondere direttamente nel body della risposta (per le richieste singole) oppure aprire uno **stream** (per esempio con Server-Sent Events) per inviare risposte e notifiche multiple nel tempo.

Streamable HTTP è pensato per funzionare bene con i load balancer, i reverse proxy e i meccanismi standard di HTTP (autenticazione, TLS, cache). Supporta anche una modalità **stateless**, in cui ogni richiesta è indipendente e il server non mantiene una sessione persistente.

1.6.3 Il vecchio trasporto HTTP+SSE

La prima versione di MCP definiva un trasporto HTTP basato su due endpoint: uno per le richieste del client e uno per lo stream di eventi del server (SSE). Questo trasporto è oggi **deprecato** in favore di Streamable HTTP, ma lo si incontra ancora in molti server esistenti e in molta documentazione. L'SDK .NET lo supporta ancora (con l'opzione `EnableLegacySse`), per compatibilità.

1.6.4 Come scegliere il trasporto

Scenario	Trasporto consigliato
Server personale, usato da un client locale sulla stessa macchina	`stdio`
Server condiviso tra più utenti o macchine	Streamable HTTP
Server in un container o dietro un proxy/reverse proxy	Streamable HTTP
Server che deve inviare eventi asincroni frequenti	Streamable HTTP
Integrazione con un IDE (VS Code, ecc.)	`stdio` o HTTP, a seconda del caso

In questo capitolo implementeremo entrambi: prima un server `stdio` completo, poi la sua versione HTTP con ASP.NET Core, in modo da padroneggiare l'intero spettro.

1.6.5 Il ruolo dell'SDK

Da sviluppatori non dobbiamo occuparci dei dettagli di framing: l'SDK .NET fornisce i trasporti già pronti. In particolare, nel pacchetto troviamo le classi `StdioServerTransport`, `StreamableHttpServerTransport` e `StreamServerTransport`, che vengono selezionate con un semplice metodo di configurazione (`WithStdioServerTransport()`, `WithHttpTransport()`). La nostra responsabilità si riduce a scegliere il trasporto giusto e a rispettare le sue regole (per esempio, non scrivere sullo stdout nei server `stdio`).

1.7 Il ciclo di vita di una sessione MCP

Una sessione MCP attraversa fasi ben precise, regolate dalla specifica. Conoscerle aiuta a capire gli errori di connessione e a progettare server robusti.

1.7.1 Fase 1: l'inizializzazione

Appena la connessione è stabilita, il client invia una richiesta `initialize` con:

- la **versione del protocollo** che intende usare;
- le proprie **capabilities**;

- le proprie informazioni (`clientInfo`: nome e versione).

Il server risponde dichiarando:

- la versione del protocollo **effettiva** (che deve essere compatibile con quella richiesta);
- le proprie **capabilities**;
- le proprie informazioni (`serverInfo`);
- facoltativamente, le **istruzioni** per il modello (un testo che l'host può includere nel prompt di sistema per spiegare come usare il server).

Le versioni del protocollo sono identificate da una data (per esempio `2025-06-18`), non seguono il versionamento semantico dei pacchetti. Client e server negoziano una versione supportata; se non è possibile concordarne una, la sessione non può essere inizializzata.

1.7.2 Fase 2: la notifica di "initialized"

Dopo aver ricevuto la risposta a `initialize`, il client invia la **notifica** `notifications/initialized`. Solo da questo momento in poi il client è autorizzato a inviare richieste "normali" (elencare i tool, invocarli, ecc.). Un server corretto deve quindi **rifiutare le richieste che arrivano prima dell'inizializzazione**. L'SDK .NET applica questa regola automaticamente.

1.7.3 Fase 3: l'operatività

In questa fase avviene il lavoro vero e proprio: discovery, invocazione di tool, lettura di risorse, recupero di prompt. Le due parti possono inoltre scambiarsi:

- `ping` per verificare che la connessione sia viva;
- notifiche di log e di cambiamento delle risorse;
- richieste di sampling (dal server verso il client).

1.7.4 Fase 4: la chiusura

La sessione termina quando:

- una delle parti chiude il trasporto (per `stdio`, quando il processo termina o lo stdin si chiude);
- il client invia una richiesta di chiusura esplicita (se il trasporto la prevede);
- scatta un timeout o un errore irreversibile.

Non esiste un metodo `shutdown` universale nella specifica base: la chiusura è, di norma, legata alla vita del trasporto. Per un server `stdio` questo significa che il processo deve uscire in modo pulito quando lo stdin viene chiuso; l'SDK gestisce anche questo per noi.

1.7.5 La sequenza riassunta

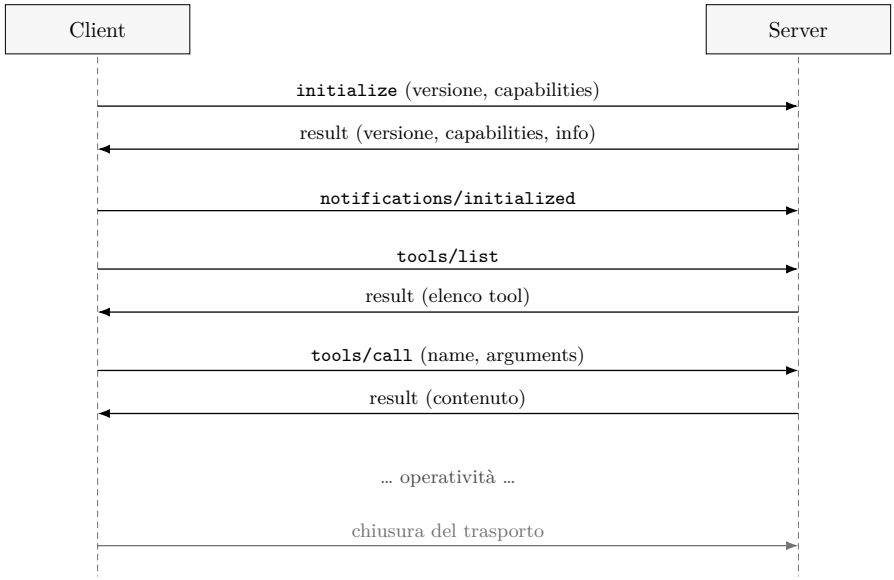


Figura 2: Sequenza completa del ciclo di vita di una sessione MCP

1.7.6 Cosa significa per il nostro codice

Tutta questa sequenza è gestita dall'SDK. La nostra unica accortezza, nel caso del trasporto `stdio`, sarà ricordare che il processo deve restare in esecuzione in attesa di messaggi, e che lo stdout è riservato al protocollo. Nelle prossime sezioni vedremo come il template `Host.CreateApplicationBuilder` + `RunAsync()` realizzi esattamente questo ciclo di vita in poche righe.

1.8 MCP vs REST API: un confronto approfondito

"Ma io ho già le mie API REST: perché dovrei usare MCP?" È la domanda più frequente, e la risposta merita una sezione intera. La chiave è capire che **non sono concorrenti**: risolvono problemi diversi e, nella maggior parte dei progetti reali, convivono.

1.8.1 Cos'è una REST API

Una REST API (Representational State Transfer) è uno stile architettonico per esporre risorse su HTTP. Un servizio definisce una serie di **endpoint** (URL) e di **verbi HTTP** (GET, POST, PUT, DELETE), con formati di richiesta

e risposta (tipicamente JSON) documentati a mano o con specifiche come OpenAPI.

Esempio:

```
GET    /api/customers/123
POST   /api/customers
PUT    /api/customers/123
DELETE /api/customers/123
```

Le REST API sono ottime per la comunicazione **tra software**: un'applicazione chiama un'altra applicazione in modo deterministico, tipizzato e prevedibile. Il chiamante sa *esattamente* quale endpoint invocare e quali parametri passare.

1.8.2 Cos'è un MCP server

Un MCP server, come abbiamo visto, espone **tool, risorse e prompt** descritti in modo autodescrittivo e *scopribili*. Il chiamante non conosce a priori gli endpoint: all'avvio chiede al server "che cosa sai fare?" e riceve un elenco di funzionalità con relative descrizioni e schemi. La decisione su *cosa* invocare spetta al **modello**, non a un programmatore che ha scritto il codice di integrazione.

1.8.3 La differenza fondamentale: chi decide cosa chiamare

	REST API	MCP server
Consumatore	Un'applicazione (codice scritto da un umano)	Un modello linguistico (o un client che lo assiste)
Scoperta	Documentazione, OpenAPI, a mano	Automatica (`tools/list`, `resources/list`, …)
Decisione di chiamata	Il programmatore, a compile time	Il modello, a runtime, in base al contesto
Contratto	Endpoint + verbi + schemi specifici per servizio	Un unico protocollo per tutti i servizi
Semantica	Legata al dominio (`/customers`, `/orders`)	Generica (tool, risorse, prompt)
Stato	Stateless (di norma)	Sessione con capabilities negoziate
Trasporto tipico	HTTP	`stdio`, Streamable HTTP

In una REST API il **programmatore** decide: scrive `GET /api/weather?city=Roma` e conosce la risposta. In un MCP server il **modello** decide: gli vengono presentati i tool disponibili ("c'è un tool `get_weather` che accetta `city`") e, quando l'utente chiede "che tempo fa a Roma?", il modello capisce da solo di dover chiamare `get_weather` con `city = "Roma"`.

1.8.4 L'autodescrizione è il vero valore

La differenza pratica più profonda è l'**autodescrizione**. Un MCP server non espone solo funzioni: espone *descrizioni* di quelle funzioni, pensate per essere lette da un modello. Il nome, la descrizione del tool e la descrizione di ogni parametro diventano parte del "prompt di sistema" che l'host costruisce per il modello. La qualità di queste descrizioni determina la probabilità che il modello usi lo strumento correttamente.

Una REST API, per contro, è descritta da una documentazione pensata per **umani** (o, nel migliore dei casi, da una specifica OpenAPI pensata per generatori di codice). Un LLM può, in teoria, imparare a usare un'API REST leggendone la documentazione, ma questo processo è fragile e va ripetuto per ogni API.

1.8.5 Il problema M×N, di nuovo

Torniamo al problema M×N. Con le REST API, ogni applicazione AI deve essere integrata a mano con ogni servizio. Con MCP, ogni servizio espone una volta la propria interfaccia standard, e *tutte* le applicazioni AI che parlano MCP possono usarla senza ulteriore lavoro. È il passaggio da una complessità moltiplicativa a una complessità additiva: M applicazioni + N server, invece di $M \times N$ integrazioni.

1.8.6 Quando usare cosa

Usa una REST API quando:

- il consumatore è un'applicazione o un servizio, non un LLM;
- hai bisogno di controllo tipizzato, prestazioni prevedibili e semantica di dominio esplicita;
- la logica di business richiede transazioni, versioning degli endpoint, contratti stabili;
- devi esporre funzionalità a terze parti con un contratto pubblico ben definito.

Usa un MCP server quando:

- il consumatore è un'applicazione basata su LLM (assistenti, agenti, copiloti);
- vuoi che il modello scopra e usi i tuoi strumenti e dati senza integrazioni ad hoc;
- vuoi scrivere l'integrazione **una volta** e riusarla con qualsiasi host (Claude, VS Code, Copilot, un tuo agente);
- vuoi esporre non solo dati, ma anche azioni e template di prompt in modo uniforme.

La combinazione vincente è, molto spesso, **entrambi**: mantieni le tue REST API come strato applicativo, e costruisci un MCP server che vi si appoggia sopra

per renderle consumabili dai modelli. È esattamente ciò che faremo nell'esempio meteo: il nostro MCP server chiamerà un'API HTTP esterna e ne esporrà il risultato come tool.

1.8.7 Un esempio concreto di differenza

Consideriamo il caso "cerca un volo". Con una REST API:

```
GET /api/flights/search?from=FCO&to=JFK&date=2026-11-01
Authorization: Bearer <token>
```

Il chiamante deve conoscere l'endpoint, i parametri, l'autenticazione e il formato della risposta. Con un MCP server, il client vede invece:

```
{
  "name": "search_flights",
  "description": "Search available flights between two airports on a
   ↪  given date.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "from": { "type": "string", "description": "Departure airport IATA
       ↪  code." },
      "to":   { "type": "string", "description": "Arrival airport IATA
       ↪  code." },
      "date": { "type": "string", "description": "Travel date in
       ↪  YYYY-MM-DD." }
    },
    "required": ["from", "to", "date"]
  }
}
```

Il modello, leggendo questa descrizione, sa *da solo* come procedere quando l'utente dice "trovami un volo da Roma a New York per il primo novembre". Non c'è codice di integrazione da scrivere: c'è solo una descrizione da leggere.

1.8.8 Tabella di sintesi operativa

Criterio	REST API	MCP server
Scopo	Comunicazione tra servizi	Fornire contesto e strumenti agli LLM
Scoperta	Manuale/OpenAPI	Automatica via protocollo
Chiamante tipico	Codice applicativo	Modello linguistico
Flessibilità d'uso	Bassa (contratto fisso)	Alta (il modello adatta gli argomenti)
Riuso tra host AI	Richiede integrazione per ciascuno	Nativamente riusabile

Criterio	REST API	MCP server
Autenticazione	API key, OAuth, ecc.	OAuth 2.1 (HTTP), trust locale (stdio)
Ecosistema .NET	ASP.NET Core, Minimal API, ecc.	`ModelContextProtocol` + `ModelContextProtocol.AspNetCore`

In sintesi: REST è il linguaggio dei **servizi**, MCP è il linguaggio dei **modelli**. Padroneggiare entrambi — e saperli comporre — è una delle competenze più richieste nello sviluppo di applicazioni AI moderne.

1.9 L'ecosistema .NET: pacchetti e namespaces

Il C# SDK ufficiale di MCP è distribuito su NuGet attraverso una famiglia di pacchetti. Conoscerla è importante per aggiungere al progetto solo ciò che serve, senza dipendenze inutili.

1.9.1 I pacchetti

Pacchetto	Scopo
`ModelContextProtocol.Core`	API di basso livello: client, server "nudo", trasporti, attributi. Dipendenze minime.
`ModelContextProtocol`	Pacchetto principale con le estensioni di *hosting* e *dependency injection*. Referenzia `Core`. È la scelta giusta per quasi tutti i progetti che non hanno bisogno di HTTP.
`ModelContextProtocol.AspNetCore`	Libreria per i server MCP basati su HTTP. Referenzia `ModelContextProtocol`.
`ModelContextProtocol.Extensions.Apps`	Estensione per costruire interfacce utente interattive renderizzate dentro gli host MCP.
`ModelContextProtocol.Extensions.Tasks`	Estensione per eseguire tool di lunga durata in modo asincrono, con polling dello stato.

Per il nostro lavoro useremo due pacchetti: `ModelContextProtocol` (per il server `stdio`) e `ModelContextProtocol.AspNetCore` (per il server HTTP).

> **Versione.** Al momento in cui scriviamo, l'ultima versione stabile è la **2.3.0**, e targetizza .NET 8, .NET 9 e .NET 10. Le API sono in evoluzione: se usi una versione diversa, verifica sempre la documentazione di riferimento (`csharp.sdk.modelcontextprotocol.io`).

1.9.2 I namespaces principali

L'SDK organizza i tipi in namespaces chiari:

- `ModelContextProtocol.Server` — i tipi lato server: gli attributi `[McpServerTool]`, `[McpServerPrompt]`, `[McpServerResource]` e i rispettivi attributi di tipo, oltre a `McpServerOptions` e `McpServer`.
- `Microsoft.Extensions.DependencyInjection` — le estensioni di configurazione: `AddMcpServer()`, `WithTools<T>()`, `WithPrompts<T>()`, `WithResources<T>()`, `WithStdioServerTransport()` e (dal pacchetto ASP.NET Core) `WithHttpTransport()`.
- `Microsoft.AspNetCore.Builder` — l'estensione `MapMcp()` per mappare l'endpoint HTTP.
- `ModelContextProtocol.Client` — i tipi lato client (`McpClient`, i trasporti client), utili per scrivere test o client .NET.
- `ModelContextProtocol.Protocol` — i tipi del protocollo a basso livello (messaggi, risultati).

1.9.3 Gli attributi di registrazione

L'SDK adotta un approccio dichiarativo: si espongono le funzionalità marcando metodi e classi con attributi. La mappatura è:

Attributo	Applicato a	Effetto
`[McpServerToolType]`	Classe	Segnala che la classe contiene dei tool
`[McpServerTool]`	Metodo	Espone il metodo come tool
`[McpServerPromptType]`	Classe	Segnala che la classe contiene dei prompt
`[McpServerPrompt]`	Metodo	Espone il metodo come prompt
`[McpServerResourceType]`	Classe	Segnala che la classe contiene delle risorse

Attributo	Applicato a	Effetto
`[McpServerResource]`	Metodo	Espone il metodo come risorsa

Le descrizioni (fondamentali, come abbiamo detto) si esprimono con l'attributo standard di .NET `[Description]`, sia sul metodo sia sui parametri. Vedremo subito esempi concreti.

1.9.4 Il modello di hosting

L'SDK si integra con il sistema di **dependency injection** di .NET: il server MCP viene registrato come servizio, i tool vengono istanziati dal container (quindi possono ricevere dipendenze nel costruttore, come un `HttpClient` o un `ILogger`), e il trasporto viene scelto con un metodo dedicato.

Questo significa che i nostri server avranno la stessa struttura di una qualunque applicazione .NET moderna: un *builder*, una configurazione dei servizi, e un `RunAsync()` finale. Non c'è nulla di esoterico: se conosci `Host.CreateApplicationBuilder` e `WebApplication.CreateBuilder`, sei già a casa.

1.9.5 Versioni e compatibilità con .NET 10

.NET 10 è la versione Long Term Support (LTS) corrente della piattaforma .NET. L'SDK MCP 2.3.0 la supporta ufficialmente (il pacchetto contiene una build per `net10.0`). Nel resto del capitolo assumeremo .NET 10, ma tutto il codice è identico su .NET 8 e .NET 9, a meno di indicazioni contrarie.

1.10 Preparare l'ambiente di sviluppo

Prima di scrivere la prima riga di codice, prepariamo l'ambiente. Il procedimento è identico su Windows, macOS e Linux; i comandi che seguono funzionano in qualsiasi terminale.

1.10.1 Verificare l'installazione di .NET

Apri un terminale e verifica la versione dell'SDK:

```
> dotnet --version
10.0.400
```

Se il comando non viene trovato, o restituisce una versione precedente alla 10, installa l'SDK dal sito ufficiale `https://dotnet.microsoft.com/download` (scegli la build SDK, non il solo runtime). Su Windows puoi anche usare il gestore `winget`:

```
> winget install Microsoft.DotNet.SDK.10
```

Verifica poi l'elenco degli SDK installati:

```
> dotnet --list-sdks
```

Se vedi la riga `10.0.x`, sei pronto. Nota: non è necessario installare Visual Studio; bastano l'SDK e un editor di testo, anche se un IDE con IntelliSense (Visual Studio Code con l'estensione C# Dev Kit, Visual Studio o Rider) renderà l'esperienza molto più comoda.

1.10.2 Creare la cartella del progetto

Creiamo una cartella per il progetto di esempio. In questo capitolo costruiremo più server; per ora ne creiamo uno per il "primo contatto":

```
> mkdir MioPrimoMcpServer
> cd MioPrimoMcpServer
```

1.10.3 Creare il progetto .NET

Per un server `stdio` useremo un semplice progetto *console*, che è il punto di partenza più leggero:

```
> dotnet new console -n MioPrimoMcpServer -o .
```

Il comando genera un progetto con un file `Program.cs` di esempio e un file di progetto `MioPrimoMcpServer.csproj`. Possiamo già dare un'occhiata al file di progetto:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

`ImplicitUsings` e `Nullable` sono già abilitati, come da template .NET moderno. Non serve altro.

1.10.4 Aggiungere i pacchetti NuGet

Aggiungiamo il C# SDK di MCP e il pacchetto di hosting generico di .NET. Aggiungiamo anche `Microsoft.Extensions.Http`, necessario per usare `AddHttpClient` nell'esempio meteo.

```
> dotnet add package ModelContextProtocol --version 2.3.0
> dotnet add package Microsoft.Extensions.Hosting
> dotnet add package Microsoft.Extensions.Http
```

Il pacchetto `ModelContextProtocol` porta con sé le API del protocollo e le estensioni di dependency injection (oltre alla dipendenza `ModelContextProtocol.Core`). Il pacchetto `Microsoft.Extensions.Hosting` serve perché i nostri server `stdio` useranno l'**host generico** (`Host.CreateApplicationBuilder`) per registrare i servizi e restare in ascolto: l'SDK, infatti, dipende solo da `Microsoft.Extensions.Hosting.Abstractions`, che non include la classe `Host`. `Microsoft.Extensions.Http` fornisce invece `AddHttpClient` per la registrazione dei client HTTP tipizzati. Non ne fissiamo qui una versione: lasciando il comando senza `--version`, NuGet seleziona una versione compatibile con il progetto `net10.0` durante il restore.

Se in futuro vorrai una versione più recente dell'SDK, puoi omettere `--version`:

```
> dotnet add package ModelContextProtocol
```

> **Attenzione.** L'SDK evolve rapidamente e alcune API cambiano tra una versione e l'altra. Per seguire questo capitolo senza sorprese, consigliamo di fissare la versione `2.3.0`, come nei comandi sopra.

1.10.5 Verificare il ripristino

Esegui una compilazione di prova per assicurarti che tutto sia in ordine:

```
> dotnet build
```

Se il comando termina con `Build succeeded` (o l'equivalente in italiano), l'ambiente è pronto. Nella prossima sezione sostituiremo il contenuto di `Program.cs` con il nostro primo server MCP.

1.10.6 Uno sguardo a ciò che abbiamo installato

Il pacchetto porta con sé, tra le altre cose, i trasporti (`StdioServerTransport` e affini), gli attributi di registrazione e le estensioni di dependency injection. Non ci sono server esterni da avviare, database da configurare o porte da aprire: un server MCP `stdio` è un normale eseguibile .NET che comunica tramite standard input/output. Questa semplicità operativa è uno dei motivi della diffusione di MCP.

1.11 Il tuo primo MCP server (stdio)

È arrivato il momento di scrivere codice. Costruiremo il server più piccolo possibile: un server `stdio` che espone un solo tool, `echo`, capace di rispondere

a un messaggio. È il classico "Hello, World!" di MCP, ma contiene già tutti gli elementi che ritroveremo, invariati, nei server più complessi.

1.11.1 Il codice completo

Sostituisci il contenuto di `Program.cs` con questo:

```
using System.ComponentModel;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using ModelContextProtocol.Server;

// 1. Crea l'host generico dell'applicazione.
var builder = Host.CreateApplicationBuilder(args);

// 2. Su stdio, lo stdout è riservato al protocollo: disattiva i log su
↪  console.
builder.Logging.ClearProviders();

// 3. Registra il server MCP, sceglie il trasporto stdio e aggancia i
↪  tool.
builder.Services.AddMcpServer()
    .WithStdioServerTransport()
    .WithTools<EchoTool>();

// 4. Avvia il server e resta in ascolto sullo standard input.
await builder.Build().RunAsync();

// 5. La classe che contiene i nostri tool.
[McpServerToolType]
public sealed class EchoTool
{
    [McpServerTool, Description("Echoes the message back to the
    ↪  client.")]
    public static string Echo(
        [Description("The message to echo.")] string message)
        => $"hello {message}";
}
```

1.11.2 Lettura riga per riga

Vediamo ogni blocco nel dettaglio.

L'host. `Host.CreateApplicationBuilder(args)` crea un *generic host*: un contenitore leggero con dependency injection, configurazione e logging. Per un server `stdio` è tutto ciò che serve; non vogliamo un web server.

Il logging. `builder.Logging.ClearProviders()` rimuove i provider di logging predefiniti, che altrimenti scriverebbero sulla console. Come abbiamo spie-

gato, in un server `stdio` lo stdout è il canale del protocollo: qualsiasi byte estraneo lo corromperebbe. È una delle trappole più comuni, e una riga la disinnesca.

La registrazione. La catena di metodi fa tre cose:

- `AddMcpServer()` registra il server MCP nel container di dependency injection;
- `WithStdioServerTransport()` seleziona il trasporto su standard input/output;
- `WithTools<EchoTool>()` registra la classe `EchoTool` come fornitrice di tool.

L'avvio. `await builder.Build().RunAsync()` costruisce l'host e lo avvia. Il server entra in ascolto sullo stdin, esegue l'handshake e risponde alle richieste finché lo stdin non viene chiuso.

Il tool. `[McpServerToolType]` marca la classe come contenitore di tool; `[McpServerTool]` espone il metodo `Echo` come tool; `[Description]` fornisce la descrizione che il modello leggerà. Il parametro `message` ha a sua volta una `[Description]`, e il metodo restituisce una stringa che l'SDK convertirà in contenuto testuale.

1.11.3 Eseguire e provare il server

Compiliamo e avviamo:

```
> dotnet run
```

Il server si mette in attesa sullo standard input e non stampa nulla (i log sono disattivati). Possiamo collaudarlo inviandogli a mano i messaggi JSON-RPC. Su una shell che supporta l'ereditarietà dello stdin possiamo incollare, una per riga, questi quattro messaggi (nel trasporto `stdio` ogni messaggio è una singola riga; qui li mostriamo su più righe solo per leggibilità — se li incolli, rimetti ciascuno su una riga):

```
{"jsonrpc":"2.0","id":1,"method":"initialize","params":{
  "protocolVersion":"2025-06-18",
  "capabilities":{},
  "clientInfo":{"name":"book-test","version":"1.0"}
}}

{"jsonrpc":"2.0","method":"notifications/initialized","params":{}}

{"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}

{"jsonrpc":"2.0","id":3,"method":"tools/call","params":{
  "name":"echo",
  "arguments":{"message":"mondo"}
}}
```

Il server risponde (sullo stdout) con la sequenza di risposte JSON-RPC, tra cui l'elenco dei tool e il risultato dell'invocazione:

```
{"jsonrpc":"2.0","id":2,"result":{"tools":[{
  "name":"echo",
  "description":"Echoes the message back to the client.",
  "inputSchema":{
    "type":"object",
    "properties":{
      "message":{"type":"string","description":"The message to echo."}
    },
    "required":["message"]
  }
}]}}
```

```
{"jsonrpc":"2.0","id":3,"result":{
  "content":[{"type":"text","text":"hello mondo"}],
  "isError":false
}}
```

1.11.4 Un dettaglio importante: il nome in minuscolo

Nota che il nome del tool è `echo`, con la iniziale minuscola, anche se il metodo C# si chiama `Echo`. L'SDK .NET, per uniformarsi alle convenzioni del protocollo, **converte i nomi dei tool in minuscolo**. È un dettaglio che conviene ricordare quando si invocano i tool a mano, in un test o in un client: si cerca `echo`, non `Echo`.

Se desideri un nome esplicito, puoi sovrascriverlo con la proprietà `Name` dell'attributo:

```
[McpServerTool(Name = "echo_message")]
[Description("Echoes the message back to the client.")]
public static string Echo([Description("The message to echo.")] string
 ↪  message)
    => $"hello {message}";
```

1.11.5 Cosa abbiamo ottenuto

Con un listato compatto abbiamo un server MCP pienamente funzionante: risponde all'handshake, si fa scoprire e invoca il tool. Tutto il resto — serializzazione, schema degli argomenti, gestione del trasporto — è opera dell'SDK. È il pattern che useremo d'ora in poi: *descrivi la funzione, lascia che l'SDK faccia il resto.*

1.12 I Tools in dettaglio

Il tool è il primitivo più usato, e merita un approfondimento. In questa sezione esamineremo tutto ciò che si può controllare nell'esposizione di un tool: nomi, descrizioni, tipi di parametro, tipi di ritorno, asincronia, annullamento e annotazioni.

1.12.1 L'anatomia di un metodo-tool

Un tool è, in definitiva, un **metodo C# marcato con `[McpServerTool]`**. L'SDK, tramite reflection, ne ricava:

- il **nome** (dal nome del metodo, normalizzato in minuscolo, o dalla proprietà `Name`);
- la **descrizione** (dall'attributo `[Description]` o dal commento XML `<summary>`);
- lo **schema di input** (dai parametri: nome, tipo .NET mappato a tipo JSON, obbligatorietà, `[Description]`);
- il **tipo di ritorno** (che determina la forma del contenuto restituito).

1.12.2 I tipi di parametro supportati

L'SDK converte i tipi .NET in tipi JSON Schema. I mapping più comuni:

Tipo .NET	Tipo JSON Schema
`string`	`string`
`int`, `long`, `short`	`integer`
`double`, `float`, `decimal`	`number`
`bool`	`boolean`
`DateTime`, `DateTimeOffset`	`string` (formato data/ora)
`enum`	`string` con `enum` dei valori
array e classi/record	`array` e `object` annidati

I parametri **opzionali** (con valore di default) non vengono inseriti nell'elenco `required` dello schema; i parametri senza default sono obbligatori. Esiste inoltre un supporto sperimentale per i tipi *nullable* di riferimento.

1.12.3 I tipi di ritorno supportati

Un tool può restituire:

- **`string`** — viene incapsulata in un contenuto di tipo testo (`{ "type": "text", "text": ... }`);
- **un oggetto, un record o un array** — viene serializzato in JSON e incapsulato **in un blocco di testo**; se inoltre si imposta `UseStructuredContent = true` sull'attributo, il risultato viene duplicato

anche nel campo `structuredContent`, che il client espone come dato già deserializzato;
- `Task<T>` / `ValueTask<T>` — per l'esecuzione asincrona;
- `IAsyncEnumerable<T>` — per restituire più risultati in modo incrementale.

Nel nostro primo server abbiamo usato `string`. Vediamo ora un esempio con un **record**, più vicino ai casi reali:

```
[McpServerTool, Description("Sums two numbers and returns a structured
 ↪  result.")]
public static AddResult Add(
    [Description("First operand.")] int a,
    [Description("Second operand.")] int b)
    => new AddResult(a, b, a + b);

public sealed record AddResult(int A, int B, int Sum);
```

Quando il client invoca `add` con `a = 2, b = 3`, il server restituisce per default un contenuto di testo contenente il JSON serializzato:

```
{ "type": "text", "text": "{\"a\":2,\"b\":3,\"sum\":5}" }
```

Se invece vogliamo che il dato viaggi anche come **contenuto strutturato** (cioè già deserializzato, senza doverlo rileggere dal testo), abilitiamo la proprietà corrispondente:

```
[McpServerTool(UseStructuredContent = true)]
[Description("Sums two numbers and returns a structured result.")]
public static AddResult Add(
    [Description("First operand.")] int a,
    [Description("Second operand.")] int b)
    => new AddResult(a, b, a + b);
```

In questo caso il client riceve sia il blocco di testo sia il campo `structuredContent` pari a `{"a":2,"b":3,"sum":5}`. La scelta dipende dal client: molti client (e modelli) si trovano bene con il testo JSON, ma se sai che il consumatore preferisce dati strutturati, `UseStructuredContent` è l'opzione giusta.

1.12.4 Tool asincroni e annullamento

I tool reali spesso eseguono I/O (chiamate HTTP, query, letture da disco). In questi casi il metodo deve essere asincrono e deve accettare un `CancellationToken`, che l'SDK associa alla cancellazione della richiesta:

```
[McpServerTool, Description("Waits for the given number of
 ↪  milliseconds.")]
public static async Task<string> Delay(
    [Description("Milliseconds to wait.")] int milliseconds,
    CancellationToken cancellationToken = default)
```

```
{
    await Task.Delay(milliseconds, cancellationToken);
    return $"waited {milliseconds} ms";
}
```

Il `CancellationToken` **non** diventa un argomento del tool: l'SDK lo riconosce come parametro speciale e lo inietta automaticamente, escludendolo dallo schema di input. Lo stesso vale per eventuali parametri di servizio risolti dal container (ci torneremo nell'esempio meteo).

1.12.5 Iniezione delle dipendenze nei tool

Le classi tool vengono istanziate dal container di dependency injection. Possiamo quindi usare il **costruttore** per ricevere i servizi, esattamente come in una qualunque classe .NET:

```
[McpServerToolType]
public sealed class TicketTool(TicketService service,
 ↪  ILogger<TicketTool> logger)
{
    [McpServerTool, Description("Creates a new support ticket.")]
    public async Task<string> CreateTicket(
        [Description("Subject of the ticket.")] string subject,
        CancellationToken cancellationToken = default)
    {
        logger.LogInformation("Creating ticket {Subject}", subject);
        var id = await service.CreateAsync(subject, cancellationToken);
        return $"ticket created with id {id}";
    }
}
```

Qui `TicketService` e `ILogger<TicketTool>` vengono iniettati nel costruttore primario, mentre `subject` è l'unico vero argomento del tool. La combinazione di *argomenti dichiarativi* e *dipendenze iniettate* è uno dei punti di forza dell'SDK .NET.

1.12.6 Le annotazioni dei tool

MCP prevede alcune **annotazioni** facoltative che aiutano il client (e il modello) a capire la natura di un tool. L'SDK le espone come proprietà dell'attributo `[McpServerTool]`:

Proprietà	Significato
`Title`	Un titolo breve e leggibile per l'interfaccia utente
`Destructive`	`true` se il tool può distruggere o alterare dati in modo irreversibile

Proprietà	Significato
`Idempotent`	`true` se chiamate ripetute con gli stessi argomenti producono lo stesso effetto
`OpenWorld`	`true` se il tool interagisce con un sistema "aperto" e mutevole
`ReadOnly`	`true` se il tool non modifica alcuno stato

Esempio:

```
[McpServerTool(Title = "Delete customer", Destructive = true, ReadOnly =
 ↪  false)]
[Description("Permanently deletes a customer and all related data.")]
public static string DeleteCustomer([Description("Customer id.")] int
 ↪  id)
    => $"customer {id} deleted";
```

Queste annotazioni non cambiano il comportamento del server, ma consentono ai client di mostrare conferme ("sei sicuro?") o di applicare politiche di sicurezza più prudenti per le operazioni distruttive.

1.12.7 Best practices

La descrizione di un tool è il suo "biglietto da visita" verso il modello. Alcune regole pratiche:

- descrivi **cosa fa** il tool, non come è implementato;
- indica **quando** usarlo e **quando no** ("usa questo tool per…", "non usarlo se…");
- descrivi ogni parametro con precisione, indicando formato e vincoli (per esempio "YYYY-MM-DD", "IATA code");
- sii coerente nel linguaggio: se un tool dice "customer", tutti i tool dovrebbero dire "customer", non alternare con "client";
- mantieni le descrizioni in inglese se il modello è usato prevalentemente in inglese; in italiano se il tuo contesto è italiano.

Una descrizione vaga produce un uso incerto del tool; una descrizione precisa è la differenza tra un agente che "ci azzecca" e uno che fallisce.

1.13 Esempio pratico: un server per il file system

Mettiamo in pratica quanto appreso costruendo un server MCP che espone operazioni sul file system: elencare i file di una cartella, leggere un file e scrivere un file. È un esempio classico, utile per capire come si traducono operazioni reali in tool.

1.13.1 Progettazione

Esporremo tre tool:

- `list_files` — elenca i file di una directory;
- `read_file` — legge il contenuto di un file di testo;
- `write_file` — scrive (o sovrascrive) un file di testo.

Per semplicità e sicurezza, tutte le operazioni saranno confinate in una **directory radice** configurabile: è una buona abitudine che limita i danni in caso di richieste errate o malevole.

1.13.2 Il codice completo

```
using System.ComponentModel;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using ModelContextProtocol;
using ModelContextProtocol.Server;

var builder = Host.CreateApplicationBuilder(args);

// Su stdio lo stdout è del protocollo: niente log su console.
builder.Logging.ClearProviders();

// Crea il tool con la sua directory radice e lo passa all'SDK.
var fileSystemTool = new FileSystemTool(rootDirectory: @"C:\dati");

builder.Services.AddMcpServer()
    .WithStdioServerTransport()
    .WithTools(fileSystemTool);

await builder.Build().RunAsync();

[McpServerToolType]
public sealed class FileSystemTool
{
    private readonly string _root;

    public FileSystemTool(string rootDirectory)
    {
        _root =
↪   Path.TrimEndingDirectorySeparator(Path.GetFullPath(rootDirectory));
        Directory.CreateDirectory(_root);
    }

    [McpServerTool, Description("Lists the files contained in a
    ↪  directory under the root.")]
    public string[] ListFiles(
```

```
        [Description("Relative directory path. Empty means the root.")]
         ↪ string relativePath = "")
    {
        var dir = Resolve(relativePath);
        return new DirectoryInfo(dir).GetFiles()
            .Select(f => f.Name)
            .OrderBy(name => name)
            .ToArray();
    }

    [McpServerTool, Description("Reads the text content of a file under
     ↪ the root.")]
    public string ReadFile(
        [Description("Relative file path.")] string relativePath)
    {
        var file = Resolve(relativePath);
        return File.ReadAllText(file);
    }

    [McpServerTool, Description("Writes text content to a file under the
     ↪ root, creating or overwriting it.")]
    public string WriteFile(
        [Description("Relative file path.")] string relativePath,
        [Description("Text content to write.")] string content)
    {
        var file = Resolve(relativePath);
        File.WriteAllText(file, content);
        return $"written {content.Length} characters to {relativePath}";
    }

    // Risolve un percorso relativo dentro la root, impedendo di
     ↪ uscirne.
    private string Resolve(string relativePath)
    {
        var full = Path.GetFullPath(Path.Combine(_root, relativePath));
        var comparison = OperatingSystem.IsWindows()
            ? StringComparison.OrdinalIgnoreCase
            : StringComparison.Ordinal;
        var rootPrefix = Path.EndsInDirectorySeparator(_root)
            ? _root
            : _root + Path.DirectorySeparatorChar;

        if (!string.Equals(full, _root, comparison) &&
            !full.StartsWith(rootPrefix, comparison))
            throw new McpException("Access denied: path escapes the root
             ↪ directory.");
        return full;
    }
}
```

1.13.3 Lettura guidata

La registrazione del servizio. Creiamo un'istanza di `FileSystemTool` con la directory radice e la passiamo direttamente a `WithTools(...)`. È la forma più semplice quando il tool ha una dipendenza che non deriva dal container (qui, una stringa di configurazione). In alternativa avremmo potuto usare l'overload `WithTools<FileSystemTool>()` e far risolvere la classe dal container, a patto di registrare prima tutti i parametri del costruttore come servizi.

La radice. Nel costruttore normalizziamo il percorso con `Path.GetFullPath` e ci assicuriamo che esista con `Directory.CreateDirectory`. Tutti i percorsi relativi ricevuti dai tool verranno risolti **dentro** questa radice.

Il metodo `Resolve`. È il cuore della sicurezza dell'esempio: combina la radice con il percorso relativo e normalizza entrambi. Accetta la radice stessa (utile per `list_files` con percorso vuoto) oppure un percorso che inizi con la radice seguita da un separatore: così `C:\dati2` non viene confusa con `C:\dati`. Il confronto ignora maiuscole e minuscole su Windows, ma è sensibile al caso sugli altri sistemi. Se il controllo fallisce, la `McpException` produce un `CallToolResult` con `isError: true`; non è un errore JSON-RPC di protocollo, che usa invece il campo `error`. `GetFullPath` normalizza il percorso ma non risolve i collegamenti simbolici: se la directory radice ne contiene, occorre verificare separatamente anche i relativi target.

I tool. `ListFiles` restituisce un array di stringhe: l'SDK lo serializza in JSON e lo incapsula in un blocco di testo (per avere anche il campo `structuredContent` basterebbe impostare `UseStructuredContent = true`). `ReadFile` e `WriteFile` restituiscono stringhe. Nota l'uso dei **valori di default** (`relativePath = ""`): rendono il parametro opzionale nello schema di input.

1.13.4 Provare il server

Avviamo il server e invochiamo i tool con una sessione JSON-RPC come visto prima. Per `list_files`:

```
{"jsonrpc":"2.0","id":5,"method":"tools/call","params":{
  "name":"list_files",
  "arguments":{"relativePath":""}
}}
```

Il server risponderà con l'elenco dei file presenti in `C:\dati`, sotto forma di array in formato JSON. Prova poi a leggere un file:

```
{"jsonrpc":"2.0","id":6,"method":"tools/call","params":{
  "name":"read_file",
  "arguments":{"relativePath":"note.txt"}
}}
```

E a scriverne uno:

```
{"jsonrpc":"2.0","id":7,"method":"tools/call","params":{
  "name":"write_file",
  "arguments":{"relativePath":"saluto.txt","content":"Ciao dal file
  ↪ system!"}
}}
```

1.13.5 Limitazioni e possibili evoluzioni

Questo esempio è volutamente essenziale. In un server reale vorresti aggiungere: la gestione di file binari (con i tipi MIME delle risorse, che vedremo più avanti), la lettura incrementale per file grandi, la gestione degli errori di I/O (file inesistente, permessi negati) con messaggi amichevoli, e la registrazione strutturata su un file di log invece che su console. Riprenderemo diversi di questi punti nelle sezioni sulle risorse e sulle best practice.

1.14 Esempio pratico: un server meteo con HTTP

Il secondo esempio mostra come un MCP server possa **integrare un'API esterna**: il tool esporrà al modello le previsioni meteo, ottenute chiamando un servizio HTTP pubblico. È il pattern "MCP sopra una REST API" che abbiamo descritto nella sezione di confronto, e introduce due concetti nuovi: l'iniezione di un `HttpClient` tipizzato e la gestione di una risposta JSON.

1.14.1 La scelta dell'API

Per l'esempio useremo **Open-Meteo** (`https://api.open-meteo.com/`), un servizio meteo gratuito che non richiede chiave API. L'endpoint che useremo è:

```
GET https://api.open-meteo.com/v1/forecast
    ?latitude=41.89&longitude=12.49&current=temperature_2m
```

La risposta è un JSON che contiene, tra le altre cose, la temperatura corrente:

```
{
  "current": {
    "temperature_2m": 18.4
  }
}
```

1.14.2 Il codice completo

Creiamo un nuovo progetto console e aggiungiamo il pacchetto, come già visto. Il server espone un unico tool, `get_temperature`, che accetta latitudine e longitudine:

```
using System.ComponentModel;
using System.Text.Json;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using ModelContextProtocol;
using ModelContextProtocol.Server;

var builder = Host.CreateApplicationBuilder(args);

// Su stdio lo stdout è del protocollo.
builder.Logging.ClearProviders();

// Registra un HttpClient tipizzato per il servizio meteo.
builder.Services.AddHttpClient<WeatherTool>(client =>
    client.BaseAddress = new Uri("https://api.open-meteo.com/"));

builder.Services.AddMcpServer()
    .WithStdioServerTransport()
    .WithTools<WeatherTool>();

await builder.Build().RunAsync();

[McpServerToolType]
public sealed class WeatherTool(HttpClient http)
{
    [McpServerTool, Description("Gets the current temperature for a
    ↪ location.")]
    public async Task<string> GetTemperature(
        [Description("Latitude of the location.")] double latitude,
        [Description("Longitude of the location.")] double longitude,
        CancellationToken cancellationToken = default)
    {
        var url = $"v1/forecast?latitude={latitude}" +
                  $"&longitude={longitude}&current=temperature_2m";
        var json = await http.GetStringAsync(url, cancellationToken);

        using var doc = JsonDocument.Parse(json);
        var temperature = doc.RootElement
            .GetProperty("current")
            .GetProperty("temperature_2m")
            .GetDouble();

        return $"Current temperature: {temperature:F1} °C";
    }
}
```

1.14.3 Lettura guidata

L'`HttpClient` tipizzato. `AddHttpClient<WeatherTool>(...)` registra nel container un `HttpClient` configurato con il base address del servizio, e lo associa alla classe `WeatherTool`. Grazie al **costruttore primario** (`WeatherTool(HttpClient http)`), l'istanza riceve automaticamente il client quando viene creata. Questo è il modo idiomatico .NET per gestire le chiamate HTTP: l'`HttpClient` è riusato, gestito e disposto correttamente dal container.

Il tool asincrono. `GetTemperature` è `async` e accetta un `CancellationToken`: come spiegato, l'SDK lo inietta automaticamente e lo esclude dallo schema di input. I veri argomenti esposti al modello sono solo `latitude` e `longitude`.

La lettura del JSON. Usiamo `System.Text.Json` (già incluso nel runtime) per estrarre la temperatura. In un'applicazione reale useresti classi fortemente tipizzate e `GetFromJsonAsync<T>`, ma la lettura manuale con `JsonDocument` rende esplicito il formato della risposta, utile a scopo didattico.

Il risultato. Il tool restituisce una stringa leggibile, che il modello potrà citare direttamente nella risposta all'utente: "La temperatura corrente è 18.4 °C".

1.14.4 Gestire gli errori dell'API esterna

Nel codice sopra, se il servizio meteo non risponde, l'eccezione di rete si propaga e l'SDK restituisce un errore generico del tool. È più elegante intercettare gli errori e restituire un messaggio chiaro. Lo faremo in modo sistematico nella sezione sulla gestione degli errori; per ora anticipiamo il concetto con un frammento:

```
try
{
    var json = await http.GetStringAsync(url, cancellationToken);
    // ... parsing ...
    return $"Current temperature: {temperature:F1} °C";
}
catch (HttpRequestException ex)
{
    throw new McpException($"Weather service unavailable:
     ↪ {ex.Message}");
}
```

Qui `McpException` (namespace `ModelContextProtocol`) segnala un errore del tool: l'SDK lo restituisce al client nel `CallToolResult` con `isError: true` e un messaggio comprensibile. Non è un errore JSON-RPC di protocollo, che viene rappresentato separatamente dal campo `error`.

1.14.5 Provare il server

Avviamo il server e invochiamo il tool per Roma (latitudine 41.89, longitudine 12.49):

```
{"jsonrpc":"2.0","id":9,"method":"tools/call","params":{
  "name":"get_temperature",
  "arguments":{"latitude":41.89,"longitude":12.49}
}}
```

Il server chiama Open-Meteo e risponde con la temperatura corrente. Il modello, da parte sua, sa quando usare questo tool perché la descrizione dichiara chiaramente cosa fa e quali parametri richiede: è l'autodescrizione di MCP in azione.

1.15 Le Risorse

Se i tool sono le "azioni" del server, le **risorse** sono i suoi "documenti": dati esposti con un URI e letti dal client (o dall'host) per arricchire il contesto del modello. Vediamo come esporle con l'SDK .NET.

1.15.1 Una risorsa semplice

Una risorsa è un metodo marcato con `[McpServerResource]`, all'interno di una classe marcata con `[McpServerResourceType]`. La proprietà `UriTemplate` definisce l'identificatore:

```
using System.ComponentModel;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using ModelContextProtocol.Server;

var builder = Host.CreateApplicationBuilder(args);
builder.Logging.ClearProviders();

builder.Services.AddMcpServer()
    .WithStdioServerTransport()
    .WithResources<AppResources>();

await builder.Build().RunAsync();

[McpServerResourceType]
public sealed class AppResources
{
    [McpServerResource(UriTemplate = "config://app", Name =
    ↪  "AppConfig")]
    [Description("Application configuration in JSON format.")]
    public static string GetConfig()
```

```
        => "{\"name\":\"demo\",\"version\":\"1.0.0\"}";
}
```

Il client può ora elencare le risorse (`resources/list`) e leggerle (`resources/read` con l'URI `config://app`). La risorsa restituisce una stringa; l'SDK la incapsula in un contenuto di testo con tipo MIME `text/plain` (personalizzabile tramite la proprietà `MimeType` dell'attributo).

1.15.2 I template di risorsa parametrizzati

Spesso una risorsa rappresenta un *insieme* di dati dello stesso tipo: un documento per ogni cliente, una pagina per ogni articolo. In questi casi si usa un **template**, in cui l'URI contiene dei segnaposto racchiusi tra graffe:

```
[McpServerResource(UriTemplate = "docs://clients/{id}", Name =
↪  "ClientDocument")]
[Description("The document of a client, identified by id.")]
public static string GetClientDocument(string id)
    => $"Document content for client {id}";
```

I segnaposto del template corrispondono ai **parametri del metodo**: quando il client legge `docs://clients/42`, l'SDK invoca `GetClientDocument("42")`. La risorsa può dichiarare un `MimeType` specifico, per esempio `application/json` o `text/markdown`:

```
[McpServerResource(UriTemplate = "docs://clients/{id}", Name =
↪  "ClientDocument", MimeType = "text/markdown")]
[Description("The document of a client, identified by id.")]
public static string GetClientDocument(string id)
    => $"# Client {id}\n\nContenuto del documento.";
```

1.15.3 Risorse e sottoscrizioni

MCP prevede che un client possa **sottoscriversi** ai cambiamenti di una risorsa (`resources/subscribe`), ricevendo una notifica quando il contenuto cambia (`notifications/resources/updated`). È la base per scenari in cui il modello deve essere aggiornato quando, per esempio, un file viene modificato o un record viene aggiornato. L'SDK .NET supporta queste operazioni tramite handler configurabili; per i nostri scopi introduttivi è sufficiente sapere che esistono.

1.15.4 Tool o risorsa?

La regola pratica per scegliere:

- se il dato deve essere **letto dal modello quando serve**, un **tool** di lettura va benissimo e spesso è più semplice;

- se il dato è **parte stabile del contesto** (configurazioni, documentazione, dati di riferimento) che il client può caricare in modo proattivo, una **risorsa** è più appropriata;
- se il dato cambia nel tempo e vuoi **notificare** i cambiamenti, le risorse con sottoscrizione sono la scelta naturale.

Nella pratica molti server espongono la stessa informazione in entrambi i modi: un tool per la lettura on-demand e una risorsa per l'accesso dichiarativo.

1.15.5 Il tipo MIME giusto

Il campo `MimeType` aiuta il client a interpretare il contenuto. Valori comuni:

Tipo MIME	Uso
`text/plain`	Testo semplice (default)
`text/markdown`	Documenti Markdown
`application/json`	Dati strutturati
`text/html`	Pagine HTML
`image/png`, `image/jpeg`	Immagini

Le risorse binarie (come le immagini) richiedono di restituire byte; l'SDK supporta anche questo caso, ma per il nostro percorso introduttivo ci concentriamo sui contenuti testuali.

1.16 I Prompt

Il terzo primitivo, il **prompt**, permette di esporre modelli di istruzioni riutilizzabili. A differenza dei tool (che eseguono codice) e delle risorse (che forniscono dati), un prompt produce **testo pronto da inviare al modello**.

1.16.1 Un prompt semplice

Come per i tool, si marca una classe con `[McpServerPromptType]` e i metodi con `[McpServerPrompt]`:

```
using System.ComponentModel;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using ModelContextProtocol.Server;

var builder = Host.CreateApplicationBuilder(args);
builder.Logging.ClearProviders();

builder.Services.AddMcpServer()
    .WithStdioServerTransport()
```

```
    .WithPrompts<AppPrompts>();

await builder.Build().RunAsync();

[McpServerPromptType]
public sealed class AppPrompts
{
    [McpServerPrompt, Description("Creates a code review prompt.")]
    public static string ReviewCode(
        [Description("The code to review.")] string code)
        => $"Please review the following code and suggest
        ↪  improvements:\n\n{code}";
}
```

Il client può elencare i prompt (`prompts/list`) e ottenerne il testo (`prompts/get`). L'utente, dall'interfaccia dell'host, seleziona il prompt `review_code` e il modello riceve l'istruzione già composta.

1.16.2 I prompt con argomenti

Un prompt può dichiarare **argomenti** con relativo schema, esattamente come un tool:

```
[McpServerPrompt, Description("Creates a meeting summary prompt.")]
public static string SummarizeMeeting(
    [Description("Title of the meeting.")] string title,
    [Description("Raw meeting transcript.")] string transcript)
    => $"Summarize the meeting '{title}' highlighting decisions and
    ↪  action items.\n\nTranscript:\n{transcript}";
```

1.16.3 Prompt o tool?

La differenza è sottile ma importante:

- un **tool** *esegue* un'operazione e restituisce un risultato (dati, conferme, contenuti calcolati);
- un **prompt** *compone* istruzioni che saranno eseguite dal modello.

Se il tuo obiettivo è "genera un testo con queste istruzioni", un prompt è la scelta giusta. Se il tuo obiettivo è "esegui questa operazione e dammi il risultato", serve un tool. In molti casi i due si combinano: un prompt può istruire il modello su *come* usare i tool disponibili.

1.16.4 Un caso d'uso reale

Immagina un team che fa spesso code review. Invece di far riscrivere ogni volta le istruzioni all'utente, il server espone un prompt `review_code`. L'utente seleziona il prompt, incolla il codice, e il modello riceve istruzioni coerenti e

complete, uguali per tutti i membri del team. Se un giorno il team decide di aggiungere un criterio di revisione (per esempio "controlla anche le dipendenze obsolete"), basta aggiornare il prompt nel server: tutti gli utenti ne beneficiano immediatamente, senza modificare alcun client.

Questa è la filosofia MCP applicata alle istruzioni: **centralizzare** ciò che è condiviso, **standardizzare** il modo di accedervi.

1.17 Server HTTP con ASP.NET Core

Finora abbiamo usato il trasporto `stdio`, ideale per un server locale lanciato dal client. Quando il server deve essere **condiviso**, raggiungibile via rete o eseguito dietro un proxy, si usa il trasporto **Streamable HTTP**, che l'SDK .NET integra direttamente con ASP.NET Core.

1.17.1 Il pacchetto aggiuntivo

Per il trasporto HTTP serve il pacchetto dedicato:

```
> dotnet add package ModelContextProtocol.AspNetCore --version 2.3.0
```

Questo pacchetto referenzia già `ModelContextProtocol`, quindi non serve aggiungerlo separatamente.

1.17.2 Il codice completo

Partiamo da un progetto ASP.NET Core *empty* (o *minimal API*), e trasformiamolo in un MCP server HTTP:

```
using System.ComponentModel;
using Microsoft.AspNetCore.Builder;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;
using ModelContextProtocol.Server;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddMcpServer()
    .WithHttpTransport()
    .WithTools<EchoTool>()
    .WithResources<AppResources>()
    .WithPrompts<AppPrompts>();

var app = builder.Build();

// Mappa l'endpoint MCP (Streamable HTTP) su /mcp.
app.MapMcp("/mcp");

await app.RunAsync();
```

```
[McpServerToolType]
public sealed class EchoTool
{
    [McpServerTool, Description("Echoes the message back to the
    ↪ client.")]
    public static string Echo([Description("The message to echo.")]
    ↪ string message)
        => $"hello {message}";
}

[McpServerResourceType]
public sealed class AppResources
{
    [McpServerResource(UriTemplate = "config://app", Name =
    ↪ "AppConfig")]
    [Description("Application configuration.")]
    public static string GetConfig() => "{\"name\":\"http-demo\"}";
}

[McpServerPromptType]
public sealed class AppPrompts
{
    [McpServerPrompt, Description("Creates a code review prompt.")]
    public static string ReviewCode([Description("The code to review.")]
    ↪ string code)
        => $"Please review the following code:\n\n{code}";
}
```

1.17.3 Le differenze rispetto a stdio

Tre sole differenze rispetto al server `stdio`:

1. usiamo `WebApplication.CreateBuilder` invece di `Host.CreateApplicationBuilder`, perché vogliamo un web server;
2. selezioniamo il trasporto con `WithHttpTransport()` invece di `WithStdioServerTransport()`;
3. mappiamo l'endpoint con `app.MapMcp("/mcp")`.

Non serve `builder.Logging.ClearProviders()`: su HTTP i log non interferiscono con il canale di comunicazione, e vederli in console è anzi utile.

1.17.4 Il percorso dell'endpoint

Il percorso passato a `MapMcp` deve corrispondere a quello usato nell'URL del client e dell'Inspector. In questo esempio l'endpoint è `/mcp`, quindi l'URL è `http://localhost:5080/mcp`:

```
app.MapMcp("/mcp");
```

Se vuoi mappare l'endpoint alla radice dell'applicazione, puoi usare `app.MapMcp()` come variante; in quel caso il client dovrà puntare a `http://localhost:5080/`.

1.17.5 Eseguire e provare

Avviamo il server specificando una porta:

```
> dotnet run --urls http://localhost:5080
```

Il server è ora un normale endpoint HTTP. Un client MCP configurato con l'URL `http://localhost:5080/mcp` potrà connettersi, eseguire l'handshake e usare tool, risorse e prompt. Nella sezione sulla configurazione dei client vedremo esattamente come indicare l'URL.

1.17.6 Streamable HTTP e modalità stateless

Come accennato, Streamable HTTP supporta una **modalità stateless**, in cui ogni richiesta è indipendente e il server non mantiene sessioni persistenti. È utile per scenari ad alta scalabilità orizzontale. L'SDK la espone tramite le opzioni del trasporto:

```
builder.Services.AddMcpServer()
    .WithHttpTransport(options => options.SessionMode =
    ↪ HttpServerSessionMode.Stateless)
    .WithTools<EchoTool>();
```

Nella modalità stateless, ogni richiesta crea un contesto server "fresco": va usata solo se i tuoi tool sono effettivamente privi di stato (nessuna sessione, nessun dato in memoria tra una chiamata e l'altra). La modalità predefinita è invece *stateful*, adatta alla maggior parte dei casi.

1.17.7 Coesistere con altre API

`MapMcp` convive senza problemi con gli altri endpoint dell'applicazione. Puoi quindi avere, nello stesso processo, sia le tue Minimal API REST sia l'endpoint MCP:

```
app.MapGet("/health", () => Results.Ok("healthy"));
app.MapMcp("/mcp");
```

È la materializzazione del principio espresso nella sezione di confronto: **REST e MCP non si escludono**, anzi spesso vivono nella stessa applicazione, ognuno con il proprio ruolo.

1.18 Configurare i client: Claude Desktop e VS Code

Un server MCP senza client è come un telefono senza interlocutore. Vediamo come collegare i nostri server ai due host più comuni: **Claude Desktop** e **Visual Studio Code** (con GitHub Copilot). Il principio è identico per qualunque altro host: si dichiara il server con un nome, un comando (per `stdio`) o un URL (per HTTP).

1.18.1 Claude Desktop con un server stdio

Claude Desktop legge la configurazione dei server da un file JSON. Su Windows si trova in:

```
%APPDATA%\Claude\claude_desktop_config.json
```

Apri il file e aggiungi il nostro server meteo nella sezione `mcpServers`:

```
{
  "mcpServers": {
    "meteo": {
      "command": "dotnet",
      "args": [
        "run",
        "--project",
        "C:\\percorso\\del\\progetto\\MeteoServer"
      ]
    }
  }
}
```

Spiegazione dei campi:

- `"meteo"` è il **nome logico** che l'host userà per riferirsi al server;
- `"command"` è l'eseguibile da lanciare (`dotnet`);
- `"args"` sono gli argomenti: `run --project <percorso>` compila ed esegue il progetto.

Dopo aver salvato, riavvia Claude Desktop: nella sua interfaccia comparirà il server (con i relativi tool) tra le integrazioni disponibili.

> **Consiglio.** In produzione è preferibile pubblicare il server (`dotnet publish`) e puntare direttamente all'eseguibile, invece di usare `dotnet run`, che compila a ogni avvio:

```
{
  "mcpServers": {
    "meteo": {
      "command": "C:\\percorso\\publish\\MeteoServer.exe",
      "args": []
```

```
    }
  }
}
```

1.18.2 Claude Desktop con un server HTTP

Se hai realizzato il server HTTP della sezione precedente, la configurazione usa l'URL invece del comando:

```
{
  "mcpServers": {
    "meteo-http": {
      "url": "http://localhost:5080/mcp"
    }
  }
}
```

1.18.3 Visual Studio Code (GitHub Copilot)

VS Code supporta i server MCP tramite il file `.vscode/mcp.json` nella root del progetto (o tramite le impostazioni). Il formato è simile:

```
{
  "servers": {
    "meteo": {
      "type": "stdio",
      "command": "dotnet",
      "args": [
        "run",
        "--project",
        "C:\\percorso\\del\\progetto\\MeteoServer"
      ]
    },
    "meteo-http": {
      "type": "http",
      "url": "http://localhost:5080/mcp"
    }
  }
}
```

Dopo aver salvato il file, VS Code/Copilot rileva il server e ne espone i tool all'agente. Potrai chiedere, per esempio: "Che tempo fa a Roma?" e osservare Copilot usare il tool `get_temperature` del tuo server.

1.18.4 Il formato comune

Come vedi, al di là di piccole differenze sintattiche, tutti gli host condividono la stessa logica:

- per `stdio`: un **comando** (e i relativi argomenti) da eseguire;
- per HTTP: un **URL** da contattare;
- opzionalmente: variabili d'ambiente, intestazioni HTTP o token di autenticazione.

È la promessa di riusabilità di MCP resa concreta: lo stesso server si collega a Claude Desktop, a VS Code e a qualunque altro host con una configurazione di poche righe, senza modificare una sola riga di codice del server.

1.18.5 Variabili d'ambiente e segreti

Molti server hanno bisogno di configurazione (chiavi API, percorsi, livelli di log). Gli host consentono di passare **variabili d'ambiente**:

```
{
  "mcpServers": {
    "meteo": {
      "command": "dotnet",
      "args": ["run", "--project", "C:\\percorso\\MeteoServer"],
      "env": {
        "API_KEY": "la-tua-chiave"
      }
    }
  }
}
```

> **Attenzione.** Non commettere mai segreti nei file di configurazione condivisi (né tantomeno nel codice sorgente). Usa variabili d'ambiente locali, un gestore di segreti o, per i server HTTP, l'autenticazione OAuth prevista dalla specifica. Torneremo sulla sicurezza tra poco.

1.19 Testare con MCP Inspector

Prima di collegare il server a un host, conviene collaudarlo in isolamento. Lo strumento ufficiale per farlo è l'**MCP Inspector**, un'applicazione web che si avvia da riga di comando e permette di: vedere i tool/risorse/prompt esposti, invocarli a mano, ispezionare i messaggi JSON-RPC e verificare l'handshake.

1.19.1 Avviare l'Inspector con un server stdio

L'Inspector si esegue con `npx` (richiede Node.js) e accetta come argomento il comando di avvio del server:

```
> npx @modelcontextprotocol/inspector dotnet run --project
 ↪  C:\percorso\MeteoServer
```

L'Inspector:

1. avvia il server come processo figlio (trasporto `stdio`);
2. apre una pagina web (di solito su `http://localhost:6274`);
3. mostra lo stato della connessione e l'esito dell'handshake.

Nella scheda **Tools** vedrai l'elenco dei tool con nome, descrizione e schema di input. Puoi selezionare un tool, compilare gli argomenti in un modulo e premere **Run** per invocarlo, osservando la risposta.

1.19.2 Avviare l'Inspector con un server HTTP

Per un server HTTP l'Inspector accetta direttamente l'URL:

```
> npx @modelcontextprotocol/inspector --transport http
 ↪  http://localhost:5080/mcp
```

1.19.3 Cosa verificare

Usa l'Inspector per controllare, in ordine:

1. **Handshake riuscito** — la connessione si stabilisce e la versione del protocollo è negoziata correttamente;
2. **Discovery** — tool, risorse e prompt compaiono con descrizioni e schemi corretti;
3. **Invocazione** — ogni tool, chiamato con argomenti validi, restituisce il risultato atteso;
4. **Errori** — chiamato con argomenti che causano un errore di esecuzione, il tool restituisce un `CallToolResult` con `isError: true` e un messaggio comprensibile (non un crash); gli errori JSON-RPC riguardano invece il protocollo;
5. **Risorse e prompt** — la lettura delle risorse e il recupero dei prompt funzionano.

1.19.4 Leggere i messaggi grezzi

L'Inspector espone anche la **vista dei messaggi**, che mostra i JSON-RPC scambiati. È il modo migliore per familiarizzare con il protocollo "dal vivo": vedrai esattamente le richieste `initialize`, `tools/list`, `tools/call` e le relative risposte, identiche a quelle che abbiamo descritto nella sezione sul JSON-RPC. Se qualcosa non torna, questa vista è il primo posto dove guardare.

1.19.5 L'Inspector come strumento di sviluppo quotidiano

Integrare l'Inspector nel flusso di lavoro ha un valore enorme: permette di iterare sul server (aggiungere un tool, migliorare una descrizione, correggere uno schema) e verificarne subito l'effetto, senza dover ogni volta riavviare un host completo. È, in pratica, il "debugger" di MCP.

Se preferisci non usare Node.js, ricorda che un server `stdio` si può collaudare anche inviando i messaggi JSON-RPC a mano sullo standard input, come abbiamo fatto nelle sezioni precedenti; l'Inspector rende però l'operazione molto più comoda e visiva.

1.20 Errori, logging e best practice

Un server MCP robusto non si limita a "funzionare quando tutto va bene": deve saper **fallire bene**. In questa sezione raccogliamo le pratiche essenziali per la gestione degli errori, il logging e la progettazione.

1.20.1 Gestire gli errori nei tool

Quando un tool incontra un errore previsto (un file inesistente, un parametro fuori intervallo, un servizio esterno irraggiungibile), non deve lasciar propagare un'eccezione grezza: deve restituire al client un **errore strutturato e comprensibile**. L'SDK offre la classe `McpException` (namespace `ModelContextProtocol`):

```
using ModelContextProtocol;

[McpServerTool, Description("Reads the text content of a file under the
 ↪  root.")]
public string ReadFile([Description("Relative file path.")] string
 ↪  relativePath)
{
    var file = Resolve(relativePath);
    if (!File.Exists(file))
        throw new McpException($"File not found: {relativePath}");
    return File.ReadAllText(file);
}
```

L'SDK intercetta la `McpException` e restituisce il risultato del tool come `CallToolResult` con `isError: true` e il messaggio fornito, che il modello può leggere e riferire all'utente. Anche un'eccezione non gestita produce un risultato di errore del tool, in genere con un messaggio generico. Questo non è un errore JSON-RPC: gli errori di protocollo (per esempio una richiesta malformata o un metodo inesistente) usano invece una risposta JSON-RPC con il campo `error`.

1.20.2 Gli errori JSON-RPC di protocollo

Gli errori JSON-RPC di protocollo hanno codici come questi, raccolti nell'enumerazione `McpErrorCode` dell'SDK:

Membro	Valore	Significato
`ParseError`	-32700	JSON non valido
`InvalidRequest`	-32600	Richiesta malformata

Membro	Valore	Significato
`MethodNotFound`	-32601	Metodo inesistente
`InvalidParams`	-32602	Parametri non validi
`InternalError`	-32603	Errore interno
`ResourceNotFound`	-32002	Risorsa inesistente
`UnsupportedProtocolVersion`	-32022	Versione del protocollo non supportata

Questi codici riguardano errori a livello di protocollo e sono distinti dagli errori di esecuzione dei tool, che vengono restituiti in `CallToolResult` con `isError: true`. Non associare quindi automaticamente a un errore di tool un codice JSON-RPC.

1.20.3 Validare gli argomenti in ingresso

Gli argomenti arrivano dal modello, e il modello può sbagliare. Valida sempre gli input critici prima di usarli:

```
[McpServerTool, Description("Divides two numbers.")]
public static double Divide(
    [Description("Dividend.")] double dividend,
    [Description("Divisor.")] double divisor)
{
    if (divisor == 0)
        throw new McpException("Cannot divide by zero.");
    return dividend / divisor;
}
```

Lo schema di input dichiara i *tipi*, ma non i *vincoli di dominio*: tocca a te verificare intervalli, formati, permessi e coerenza.

1.20.4 Il logging: dove e come

Abbiamo già visto che nei server `stdio` il logging su console va disattivato, perché lo stdout è il canale del protocollo. La chiamata che lo fa è una sola:

```
builder.Logging.ClearProviders();
```

Ma allora come si osservano i log? Le opzioni:

- **log su file** — registra i log su un file, non sulla console;
- **log su stderr** — tecnicamente lo stderr non è usato dal protocollo `stdio`; alcuni preferiscono dirottarvi i log;
- **MCP logging** — il protocollo prevede un meccanismo per inviare messaggi di log al client (`logging/setLevel` e notifiche di log), supportato dall'SDK.

La regola d'oro, per `stdio`, è una sola: **mai scrivere sullo stdout** nulla che non sia un messaggio JSON-RPC.

1.20.5 Dipendenze e testabilità

Progettare i tool come classi con dipendenze iniettate (come `WeatherTool` con il suo `HttpClient`) ha un beneficio collaterale prezioso: la **testabilità**. In un test unitario puoi sostituire l'`HttpClient` con un handler fittizio, oppure iniettare un'interfaccia `IWeatherService` e testare il tool in isolamento, senza rete né processi esterni. I metodi tool sono, in fondo, normali metodi C#: si testano come qualsiasi altro metodo.

1.20.6 Naming e coerenza

- Usa nomi di tool **chiari e verbosi**: `search_flights` è meglio di `sf`.
- Ricorda che l'SDK **converte i nomi in minuscolo**: scegli nomi già in stile `snake_case` per evitare sorprese (oppure usa la proprietà `Name` per un controllo esplicito).
- Mantieni **coerenza terminologica** tra tool, descrizioni e parametri.
- Esponi **un concetto per tool**: un tool che fa troppe cose è difficile da descrivere e da usare correttamente.

1.20.7 Piccole cose che fanno la differenza

- Descrivi i **valori di ritorno** nelle descrizioni, non solo gli input.
- Segnala nelle descrizioni i **casi limite** ("se il file non esiste, restituisce un errore").
- Usa le **annotazioni** (`ReadOnly`, `Destructive`, `Idempotent`) per informare i client sulla natura del tool.
- Mantieni i tool **idempotenti** quando possibile: rende il comportamento del modello più prevedibile e sicuro.
- Versiona i tuoi server (`serverInfo.name` e `version`) per tracciare le distribuzioni.

Queste pratiche, sommate, trasformano un server "che funziona" in un server **affidabile**, che un modello può usare con fiducia e che un team può mantenere nel tempo.

1.21.1 Risoluzione dei problemi comuni

Durante lo sviluppo di un server MCP si incontrano, quasi immancabilmente, alcuni problemi ricorrenti. Raccogliamo qui i più comuni, con la causa e la soluzione.

Il server "non risponde" (stdio)

Sintomo. Il client si connette ma non riceve risposte valide; i messaggi sembrano corrotti.

Causa più probabile. Qualcosa scrive sullo **stdout** del server, inquinando il flusso JSON-RPC. Nei server creati con `Host.CreateApplicationBuilder`, i provider di logging predefiniti scrivono sulla console.

Soluzione. Disattiva il logging su console:

```
builder.Logging.ClearProviders();
```

Se hai bisogno di log, scrivili su file o su stderr, mai sullo stdout.

"Unknown tool: 'X'"

Sintomo. Il client elenca correttamente il tool, ma l'invocazione fallisce con un errore del tipo `Unknown tool: 'Echo'`.

Causa. L'SDK .NET **converte i nomi dei tool in minuscolo**: il metodo `Echo` viene esposto come `echo`. Se il client (o il tuo test a mano) invoca `Echo` con la maiuscola, il server non lo riconosce.

Soluzione. Invoca sempre il nome in minuscolo, oppure imposta un nome esplicito con la proprietà `Name`:

```
[McpServerTool(Name = "echo_message")]
[Description("Echoes the message back to the client.")]
public static string Echo(string message) => $"hello {message}";
```

"MCP server process exited unexpectedly"

Sintomo. Il client (o l'Inspector) segnala che il processo del server è terminato subito dopo l'avvio, senza messaggi.

Cause possibili.

1. Il server **va in crash all'avvio** (un'eccezione nel costruttore di un tool, una dipendenza non registrata, un percorso inesistente).
2. Il **comando di avvio è sbagliato** (percorso dell'eseguibile o della DLL non corretto).
3. Il server, per qualche motivo, **termina immediatamente** (per esempio se eseguito senza lo stdin collegato).

Diagnosi. Esegui il server **a mano** nel terminale e osserva se parte e resta in attesa, oppure se stampa un errore e termina. Per un server `stdio`, lanciarlo a mano non produce output (i log sono disattivati) ma il processo deve restare vivo:

```
> dotnet run
```

Se il processo termina subito, c'è un errore di avvio: controlla lo stack trace (temporaneamente, puoi riattivare i log su file per vedere l'eccezione).

Errore di versione del protocollo (-32022)

Sintomo. Durante l'handshake il server risponde con un errore di versione non supportata.

Causa. Il client richiede una versione del protocollo che il server (o l'SDK) non supporta.

Soluzione. Verifica la versione del protocollo supportata dall'SDK e dal client che stai usando, quindi aggiorna uno dei due a una versione compatibile. I client meno recenti potrebbero non supportare la versione richiesta dal server.

"Porta già in uso" (server HTTP)

Sintomo. All'avvio del server HTTP, `System.IO.IOException: address already in use`.

Soluzione. Cambia porta con `--urls`:

```
> dotnet run --urls http://localhost:5081
```

E aggiorna di conseguenza l'URL nella configurazione del client o dell'Inspector.

L'Inspector non si connette

Sintomo. L'Inspector si avvia ma non riesce a parlare con il server.

Controlli in ordine:

1. il comando passato all'Inspector avvia davvero il server (provalo a mano);
2. per `stdio`, il server non scrive nulla sullo stdout;
3. per HTTP, l'URL è corretto e il server è in ascolto sulla porta giusta;
4. non ci sono firewall o proxy che bloccano la connessione (per i server HTTP locali, di solito non è un problema).

"API non trovata" dopo un aggiornamento

Sintomo. Il codice che compilava non compila più, con errori su `AddMcpServer`, `WithTools` o sugli attributi.

Causa. L'SDK MCP evolve rapidamente e alcune API cambiano tra versioni.

Soluzione. Fissa la versione del pacchetto nel file di progetto (per esempio `--version 2.3.0`), come consigliato in questo capitolo, e consulta la documentazione di riferimento (`csharp.sdk.modelcontextprotocol.io`) quando aggiorni.

Metodo di diagnosi universale

Quando qualcosa non va, riduci il problema al minimo: un server con un solo tool, un solo client, un solo trasporto. Verifica l'handshake, poi la discovery,

poi l'invocazione. Quasi sempre, isolando i tre passi, la causa emerge da sola. E ricorda la vista dei messaggi grezzi dell'Inspector: è la radiografia di ogni sessione MCP.

1.21.2 Scrivere un client .NET

Finora abbiamo costruito server. Ma lo stesso SDK offre anche il lato **client**, utile per scrivere test automatici, per creare i nostri host o per integrare un server MCP dentro un'applicazione .NET. Concludiamo il capitolo con un assaggio, usando il codice che abbiamo già verificato funzionante.

1.21.2.1 I pacchetti lato client

Il client vive nel pacchetto principale `ModelContextProtocol` (le API di basso livello stanno nel pacchetto `Core`, già referenziato da questo). Non serve nulla di più:

```
> dotnet add package ModelContextProtocol --version 2.3.0
```

1.21.2.2 Connettere un client a un server stdio

Il client avvia il server come processo figlio e comunica tramite `StdioClientTransport`:

```
using ModelContextProtocol.Client;
using ModelContextProtocol.Protocol;

// Il server viene lanciato come processo figlio.
var transport = new StdioClientTransport(new StdioClientTransportOptions
{
    Command = "dotnet",
    Arguments = new List<string> {
 ↪  @"C:\percorso\EchoServer\EchoServer.dll" },
    WorkingDirectory = @"C:\percorso\EchoServer"
});

await using var client = await McpClient.CreateAsync(transport, new
 ↪  McpClientOptions
{
    ClientInfo = new() { Name = "book-client", Version = "1.0" }
});

Console.WriteLine("Tool disponibili:");
foreach (var tool in await client.ListToolsAsync())
{
    Console.WriteLine($"  - {tool.Name}: {tool.Description}");
}
```

Punti chiave:

- `StdioClientTransportOptions.Command` è l'eseguibile da lanciare (`dotnet`), e `Arguments` contiene il percorso della DLL del server (l'output di `dotnet publish` o di `dotnet build`);
- `McpClient.CreateAsync` esegue l'handshake e restituisce un client pronto;
- `ClientInfo` identifica il nostro client verso il server;
- `await using` garantisce la corretta chiusura della sessione e del processo.

1.21.2.3 Invocare un tool

Una volta connessi, invocare un tool è immediato:

```
var result = await client.CallToolAsync("echo", new Dictionary<string,
 ↪  object?>
{
    ["message"] = "mondo"
});

Console.WriteLine(result.Content.FirstOrDefault()?.Text);
```

`CallToolAsync` restituisce un `CallToolResult` con la lista dei contenuti: per un tool che restituisce testo, il primo elemento è un blocco di tipo `text` con la proprietà `Text`. Ricorda di usare il **nome in minuscolo** del tool, per la convenzione che abbiamo già incontrato.

1.21.2.4 Connettere un client a un server HTTP

Per un server HTTP si usa invece `HttpClientTransport`:

```
using ModelContextProtocol.Client;

var transport = new HttpClientTransport(new HttpClientTransportOptions
{
    Endpoint = new Uri("http://localhost:5080/mcp")
});

await using var client = await McpClient.CreateAsync(transport, new
 ↪  McpClientOptions
{
    ClientInfo = new() { Name = "book-client", Version = "1.0" }
});
```

Il resto del codice è identico: `ListToolsAsync`, `CallToolAsync`, `ListResourcesAsync`, `ReadResourceAsync`, `ListPromptsAsync`, `GetPromptAsync`. L'uniformità tra trasporti è uno dei frutti del protocollo: cambi canale di comunicazione senza cambiare il codice applicativo.

1.21.2.5 Un client per i test

Il client è lo strumento ideale per i **test di integrazione** di un server MCP: puoi avviare il server, invocare i tool con argomenti noti e verificare i risultati, il tutto in un normale test .NET. È un pattern che consigliamo caldamente: i test di integrazione via client catturano problemi che i soli test unitari sui metodi non vedono (handshake, schemi, serializzazione, trasporto).

1.21.2.6 Client e server nello stesso ecosistema

Con questa sezione il cerchio si chiude: con gli stessi pacchetti abbiamo costruito **server** (stdio e HTTP), **risorse**, **prompt** e ora un **client**. L'SDK .NET copre l'intero spettro del protocollo, e averlo esplorato per intero è la base migliore per i capitoli successivi.

1.21 Sicurezza e operatività

Un MCP server è un **ponte tra un modello e i tuoi sistemi**: se non è protetto, diventa un ponte per chiunque riesca a raggiungerlo. La sicurezza va considerata fin dall'inizio, non aggiunta alla fine.

Il principio del minimo privilegio

Il principio guida è semplice: **esponi solo ciò che serve, con i permessi minimi necessari**. Se il server deve leggere file, non deve poter scrivere ovunque; se deve interrogare un database, usi un utente con soli diritti di lettura. Ogni tool in più, ogni permesso in più, è una superficie d'attacco in più.

Nel nostro esempio file system abbiamo applicato questo principio confinando tutte le operazioni in una directory radice. Il costruttore normalizza `_root` con `Path.GetFullPath` e rimuove gli eventuali separatori finali con `Path.TrimEndingDirectorySeparator`. In `Resolve`, il percorso normalizzato è ammesso solo se coincide con la radice o inizia con la radice seguita da un separatore: così un percorso come `C:\dati2` non supera il controllo per una root `C:\dati`. Il confronto ignora maiuscole e minuscole su Windows ed è sensibile al caso sugli altri sistemi.

```
var full = Path.GetFullPath(Path.Combine(_root, relativePath));
var comparison = OperatingSystem.IsWindows()
    ? StringComparison.OrdinalIgnoreCase
    : StringComparison.Ordinal;
var rootPrefix = Path.EndsInDirectorySeparator(_root)
    ? _root
    : _root + Path.DirectorySeparatorChar;

if (!string.Equals(full, _root, comparison) &&
    !full.StartsWith(rootPrefix, comparison))
    throw new McpException("Access denied: path escapes the root
    ↪  directory.");
```

La radice stessa è ammessa, così un percorso relativo vuoto può indicare la directory radice. Questa difesa contro il **path traversal** (un argomento come `..\..\Windows`) è essenziale in qualunque tool che tocchi il file system. `GetFullPath` normalizza i percorsi ma non risolve i collegamenti simbolici: se la directory radice ne contiene, occorre verificare separatamente anche i relativi target.

La minaccia della prompt injection

Il modello è, per sua natura, influenzabile dal testo che riceve. Un documento malevolo letto dal server, o un argomento fornito da un utente, potrebbe contenere istruzioni nascoste che spingono il modello a compiere azioni indesiderate (esfiltrare dati, eseguire tool pericolosi). È la cosiddetta **prompt injection**.

Le contromisure principali:

- **separa i dati dalle istruzioni**: quando inietti nel contesto il contenuto di una risorsa o il risultato di un tool, delimitalo chiaramente e istruisci il modello a trattarlo come dato, non come comando;
- **convalida e igienizza** gli input prima di usarli;
- **limita i tool pericolosi**: le operazioni distruttive richiedono conferma esplicita dell'utente (molti host lo fanno automaticamente se il tool è annotato `Destructive = true`);
- **non esporre segreti** nei dati che il server restituisce.

Autenticazione e autorizzazione

- Per i server `stdio`, il trust è locale: il server gira come processo del client e, di norma, eredita i permessi dell'utente. La protezione è data dall'ambiente, non dal protocollo.
- Per i server **HTTP**, la specifica prevede **OAuth 2.1** per autenticare i client. Esporre un server MCP HTTP su Internet senza autenticazione è quasi sempre un errore: chiunque potrebbe usarne i tool.
- L'SDK ASP.NET Core offre primitive per l'autenticazione (`AddAuthorizationFilters`) e si integra con i meccanismi standard di ASP.NET Core (policy, middleware).

Segreti e configurazione

Non scrivere mai chiavi API, password o token nel codice sorgente o nei file di configurazione condivisi. Usa:

- **variabili d'ambiente** (per gli host che le supportano);
- **user secrets** in sviluppo (`dotnet user-secrets`);
- un **gestore di segreti** (Azure Key Vault, AWS Secrets Manager, ecc.) in produzione;

- per i server HTTP, i meccanismi di autenticazione del protocollo.

Controllare l'esposizione di rete

Un server HTTP MCP espone un endpoint: applicalo le stesse regole di qualunque servizio web. In particolare:

- usa **TLS** (HTTPS) in produzione;
- **limita l'accesso** con firewall, reti private o VPN;
- **registra** le richieste (senza dati sensibili) per poter indagare eventuali abusi;
- tieni **aggiornati** i pacchetti: l'SDK MCP evolve e corregge vulnerabilità.

Checklist di sicurezza

Prima di mettere in produzione un server MCP, verifica:

☐ ogni tool applica il minimo privilegio;
☐ gli input sono validati e i percorsi sono confinati;
☐ i segreti non sono nel codice né nella configurazione condivisa;
☐ i tool distruttivi sono annotati e richiedono conferma;
☐ i server HTTP usano TLS e autenticazione;
☐ i log non contengono dati sensibili;
☐ le dipendenze sono aggiornate.

La sicurezza non è un punto di arrivo, ma un processo: rivedi questa checklist a ogni modifica rilevante del server.

1.22 Riepilogo, glossario ed esercizi

Siamo arrivati alla fine del capitolo. Ripercorriamo il cammino fatto e consolidiamo i concetti con un glossario e alcuni esercizi.

1.22.1 Riepilogo

In questo capitolo abbiamo:

- capito **perché** i modelli, da soli, sono isolati e perché servono strumenti, dati e istruzioni;
- introdotto il **Model Context Protocol** come standard aperto che risolve il problema M×N delle integrazioni;
- esaminato l'**architettura** a tre ruoli (host, client, server) e la negoziazione delle capabilities;
- studiato i **primitivi** (tool, risorse, prompt) e i primitivi secondari;
- letto il **JSON-RPC 2.0** e i metodi standard del protocollo;
- confrontato **MCP e REST API**, concludendo che sono complementari, non alternativi;

- esplorato i **trasporti** (`stdio` e Streamable HTTP) e il **ciclo di vita** di una sessione;
- configurato l'ambiente **.NET 10** e aggiunto i pacchetti ufficiali del C# SDK;
- implementato e collaudato un **primo server `stdio`**;
- approfondito i **tool** (parametri, ritorni, asincronia, DI, annotazioni);
- costruito due esempi completi: un server per il **file system** e un server **meteo**;
- esposto **risorse** e **prompt**;
- realizzato un **server HTTP** con ASP.NET Core;
- configurato i **client** (Claude Desktop, VS Code) e usato l'**MCP Inspector**;
- raccolto le **best practice** di errori, logging e **sicurezza**, insieme a una guida alla **risoluzione dei problemi comuni**;
- scritto anche un **client .NET** per consumare i server MCP nei nostri programmi e test.

1.22.2 Glossario

Termine	Definizione
MCP	Model Context Protocol: protocollo aperto che standardizza la fornitura di contesto agli LLM
Host	L'applicazione che ospita il modello e orchestra i client (es. Claude Desktop, VS Code)
Client	Componente che gestisce la connessione con un singolo server MCP
Server	Processo che espone tool, risorse e prompt
Tool	Funzione eseguibile dal modello su richiesta
Resource	Dato esposto con un URI, letto dal client per il contesto
Prompt	Modello di istruzioni riutilizzabile
Capabilities	Capacità dichiarate e negoziate tra client e server
JSON-RPC 2.0	Il formato di messaggi su cui si basa MCP
stdio	Trasporto su standard input/output
Streamable HTTP	Trasporto di rete raccomandato dalla specifica moderna
Handshake	La fase di inizializzazione (`initialize` + `notifications/initialized`)
Structured content	Contenuto strutturato restituito da un tool (oltre al testo)

1.22.3 Esercizi proposti

1. **Estendi il server file system.** Aggiungi un tool `delete_file` che elimina un file dentro la root, annotandolo `Destructive = true`, e un tool `list_directories` che elenca le sottocartelle.
2. **Schema personalizzato.** Aggiungi al server meteo un parametro `units` di tipo `enum` (`Celsius`, `Fahrenheit`) e usa il valore per scegliere il parametro dell'API.
3. **Una risorsa dinamica.** Esponi come risorsa `stats://filecount` il numero di file presenti nella directory radice, usando un metodo che esegue il conteggio al momento della lettura.
4. **Un prompt con argomenti.** Crea un prompt `commit_message` che, dati un elenco di modifiche e un contesto, produca l'istruzione per scrivere un buon messaggio di commit.
5. **Versione HTTP.** Converti il server file system in un server HTTP con `MapMcp("/files")` e collaudalo con l'MCP Inspector.
6. **Gestione errori.** Rendi `ReadFile` capace di distinguere tra file inesistente (errore chiaro) e permessi negati (errore chiaro), senza far mai crashare il server.
7. **Test unitario.** Scrivi un test per il tool `Divide` che verifichi il caso di divisione per zero.
8. **Configurazione.** Collega il server meteo a Claude Desktop e chiedi "che tempo fa a Milano?" (latitudine 45.46, longitudine 9.19).

1.22.4 Nel prossimo capitolo

Nel prossimo capitolo costruiremo su queste fondamenta e approfondiremo l'integrazione con i dati, realizzando un **server MCP che interagisce con un database** con Entity Framework Core.